



CPE3243 วิศวกรรมซอฟต์แวร์ (Software Engineering)

Piyavit Laung-Aram
Major of Computer Engineering
Faculty of Engineering
Ramkhamhaeng University, Thailand

The left side of the slide features a series of overlapping, semi-transparent geometric shapes, primarily triangles and parallelograms, in shades of gray. These shapes are arranged in a way that creates a sense of depth and movement, pointing towards the right.

DevOps



DevOps

DevOps คือ รูปแบบการทำงานที่ทำให้นักพัฒนาซอฟต์แวร์ (Software Developer) และคนวางโครงสร้างพื้นฐาน (infrastructure) ของระบบ (Operation) ทำงานร่วมกัน โดยมีเป้าหมายที่จะให้ระบบงานที่พัฒนาขึ้นสามารถใช้งานได้อย่างรวดเร็ว



DevOps(2)

ระเบียบวิธีการพัฒนาการทำงานในอุตสาหกรรมซอฟต์แวร์ และอุตสาหกรรมไอที ที่ใช้เป็นแนวทางปฏิบัติและเป็นเครื่องมือในการทำงาน โดยจะผสมผสานการทำงานร่วมกัน ทำให้งานการพัฒนาซอฟต์แวร์ (Dev) และการดำเนินงานด้านไอที (Ops) เป็นแบบอัตโนมัติสอดคล้องกัน เพื่อเป็นแนวทางในการปรับปรุงและลดวงจรชีวิตในการพัฒนาระบบให้สั้นลง เพื่อให้ระบบที่พัฒนาขึ้นสามารถใช้งานได้จริงอย่างรวดเร็ว



DevOps (3)

- ปกติ Develop Team จะมีเป้าหมายที่ความรวดเร็วในการพัฒนาและส่งผลงานให้ตรงตามเวลาที่กำหนด
- ปกติ Operation Team จะมีเป้าหมายที่ความเสถียรของระบบเป็นสำคัญ

DevOps คือการผสมผสานการทำงานของทั้ง 2 ทีมเข้าด้วยกัน



DevOps Engineer

อาชีพใหม่ที่มาแรงมากในปัจจุบัน ที่ดีมาคู่กับ Data Engineer

คือ DevOps Engineer

- มีหน้าที่ที่เข้ามาช่วยเชื่อมและประสานการพัฒนาซอฟต์แวร์ (Dev) และการดำเนินงานด้านไอที (Ops) เข้าด้วยกัน
- มีหน้าที่ระหว่าง Development และ Operation
- เป็นคนที่สามารถดูแล Infrastructure ,สร้าง Environment , Deploy Monitor และแก้ปัญหาต่าง ๆ ที่เกิดขึ้นบนระบบซอฟต์แวร์ที่ใช้งานจริง

หน้าที่เหล่านี้สอดคล้องกับวิธีการพัฒนาซอฟต์แวร์แบบ Agile



DevOps Engineer(2)

ทักษะที่ DevOps Engineer ต้องมี

DevOps Engineer Skills

Linux-Based Command

Version Control

Programming

Cloud Engineering



DevOps Engineer(3)

ทักษะที่มีทั้ง 4 จะถูกใช้ในการ นำระบบซอฟต์แวร์ที่พัฒนาขึ้น เข้าสู่สภาวะ
แวดล้อมในการทำงานจริงได้อย่างรวดเร็วทั้งแบบแมนนวลและแบบ
อัตโนมัติ เช่น การพัฒนาซอฟต์แวร์บน Docker Container และนำ
ระบบงานที่พัฒนาขึ้นบน Cloud Server โดยใช้แนวทาง CI/CD Pipeline
ซึ่ง Pipeline จะถูกกระตุ้นให้เริ่มทำงานเมื่อมีการ Push Source Code ไปยัง
Version Control เช่น Gitlab Github ฯ



CI/CD คืออะไร?

Continuous Integration (CI) คือกระบวนการรวบรวมโค้ดที่ได้รับการพัฒนาจากสมาชิกแต่ละคนในทีมให้เป็นขึ้นเดียว จากนั้นทำการทดสอบด้วย Test Script เพื่อตรวจสอบว่าโค้ดแต่ละส่วนเข้ากัน ไม่เกิดข้อผิดพลาดก่อนจะรวมโค้ดไปยัง Branch Main หรือ Master

หลังจากขั้นตอนการทำ CI แล้ว เราจะดำเนินการ Continuous Delivery และ Continuous Deployment (CD) เป็นกระบวนการส่งออกโค้ดหรือระบบ ที่พัฒนาขึ้นไปยังระบบจริง หรือที่เรียกว่า Production โดยการทำงานของ CD จะเป็นการทำงานแบบอัตโนมัติ



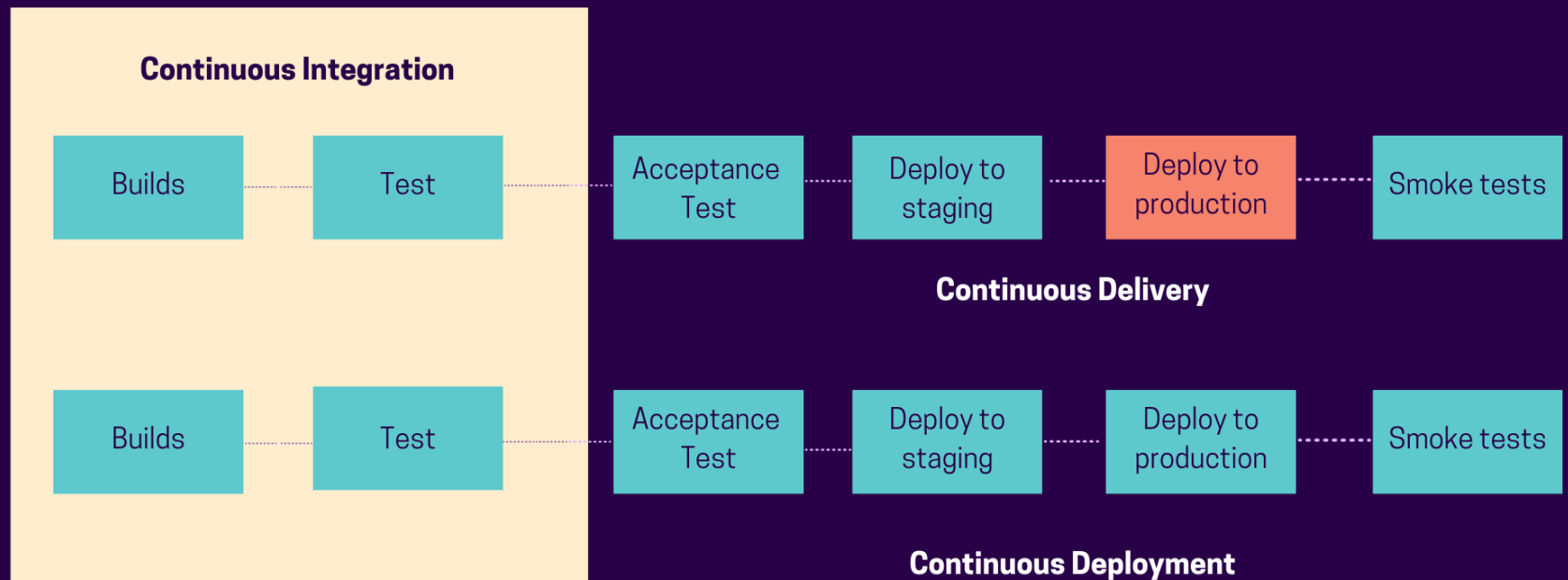
CI/CD คืออะไร? (2)

หากสรุปโดยง่าย CI/CD คือขั้นตอนการทำงานที่ครอบคลุมตั้งแต่การรวบรวมโค้ดจากสมาชิกในทีม, การทดสอบความเข้ากันได้ของโค้ด, และการส่งออกโค้ดไปยัง Production แบบอัตโนมัติ



CI/CD คืออะไร? (3)

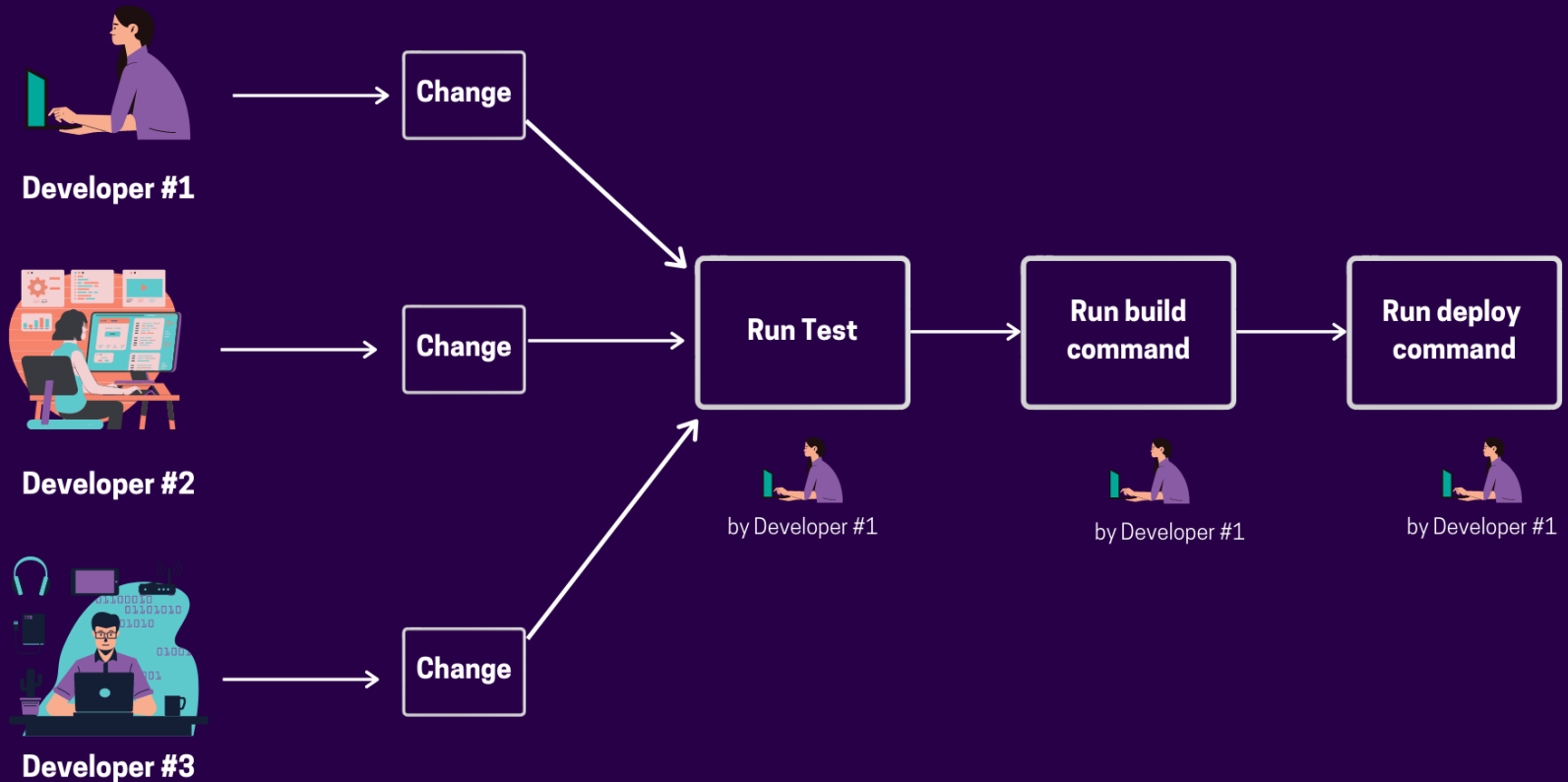
- Automated Process
- Manual Process





ก่อนมี CI/CD

Before CI/CD





ก่อนมี CI/CD (2)

แต่ก่อนเมื่อโปรแกรมเมอร์ในทีมแต่ละคนเพิ่มโค้ดเข้ามา จะต้องมีการ
โปรแกรมเมอร์ 1 คน จัดการ Run Command ต่างๆ แบบทำมือ ซึ่งในแต่ละ
วันโปรแกรมเมอร์ Commit Code มากกว่า 1 ครั้ง โปรแกรมเมอร์คนที่ต้อง
รวบรวมของทุกคนจะต้องทำกระบวนการนั้นซ้ำซ้อน (Repetitive) ส่งผลให้
โปรแกรมเมอร์ไม่มีความสุขในการทำงาน เนื่องจากเป็นการทำงานที่น่าเบื่อ
เราอาจกล่าวได้ว่า การทำงานในรูปแบบนี้คือการทำงานรูปแบบเดิมๆ ไม่มี
อะไรใหม่ ดังนั้นการนำ ขั้นตอนการทำงานโดยอัตโนมัติ หรือ Automated
Process มาร่วมกับการทำงานผ่าน CI/CD จึงตอบโจทย์มากกว่า



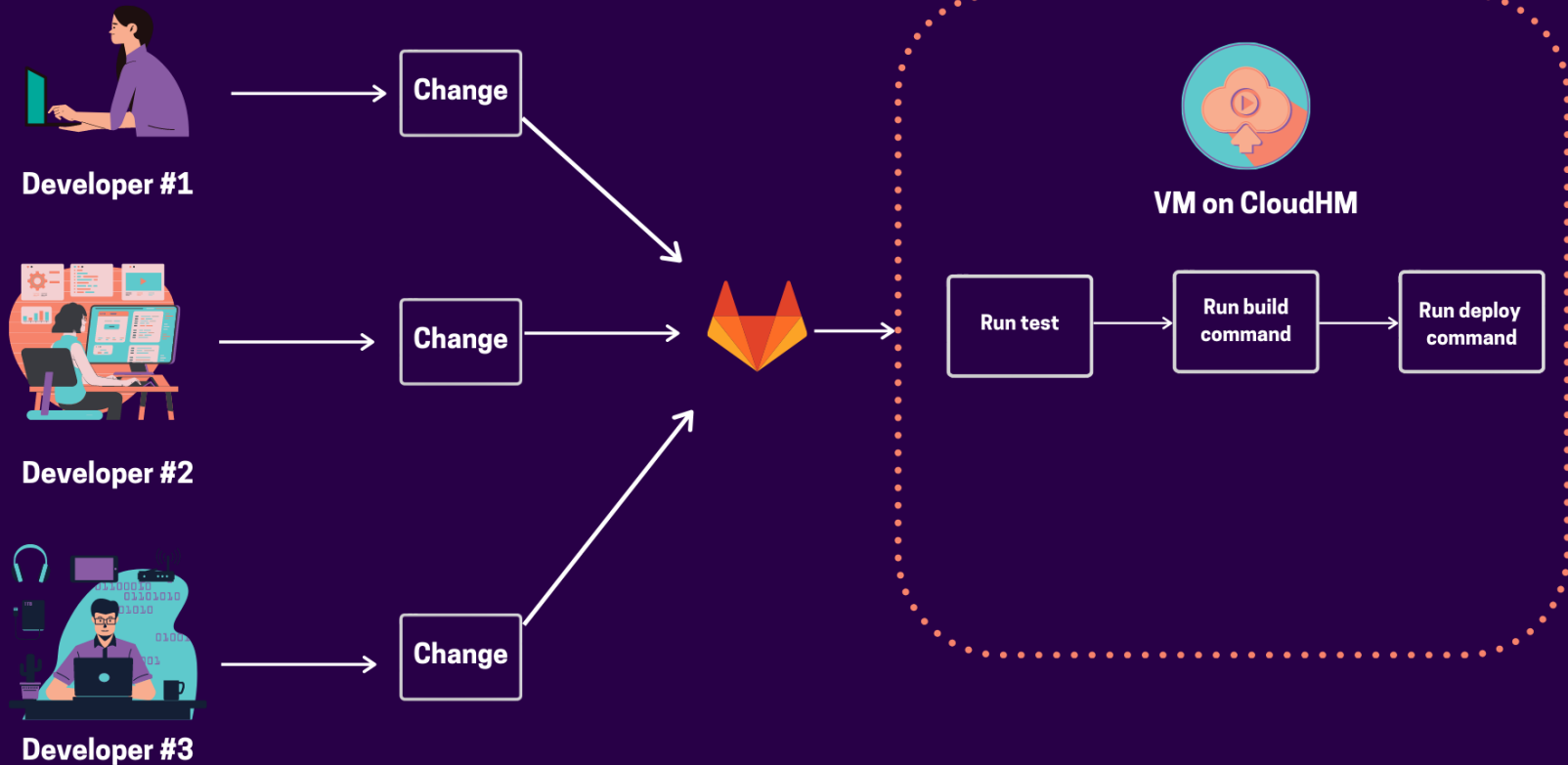
ก่อนมี CI/CD (3)

- การทำงานผ่าน CI/CD จะช่วยลดระยะเวลาการทำงานในส่วนที่ไม่จำเป็น ประหยัดเวลาการทำงาน
- สนับสนุนให้โปรแกรมเมอร์ทำงานส่วนอื่นที่ก่อเกิดประโยชน์ให้มากกว่า
- พัฒนารูปแบบการทำงานให้มีผลลัพธ์ที่มีประสิทธิภาพมากขึ้น กระบวนการ CI/CD เข้ามา
- ช่วยตรวจสอบการทำงานของระบบ ข้อผิดพลาดต่างๆที่อาจเกิดขึ้น ตั้งแต่ขั้นตอนการพัฒนา ระบบ จึงทำให้มั่นใจได้ว่าโค้ดทั้งหมดที่จะส่งไปยังระบบจริง มีคุณภาพและเป็นไปตามมาตรฐานที่คาดหวังไว้



การทำงานแบบ CI/CD

With GITLAB



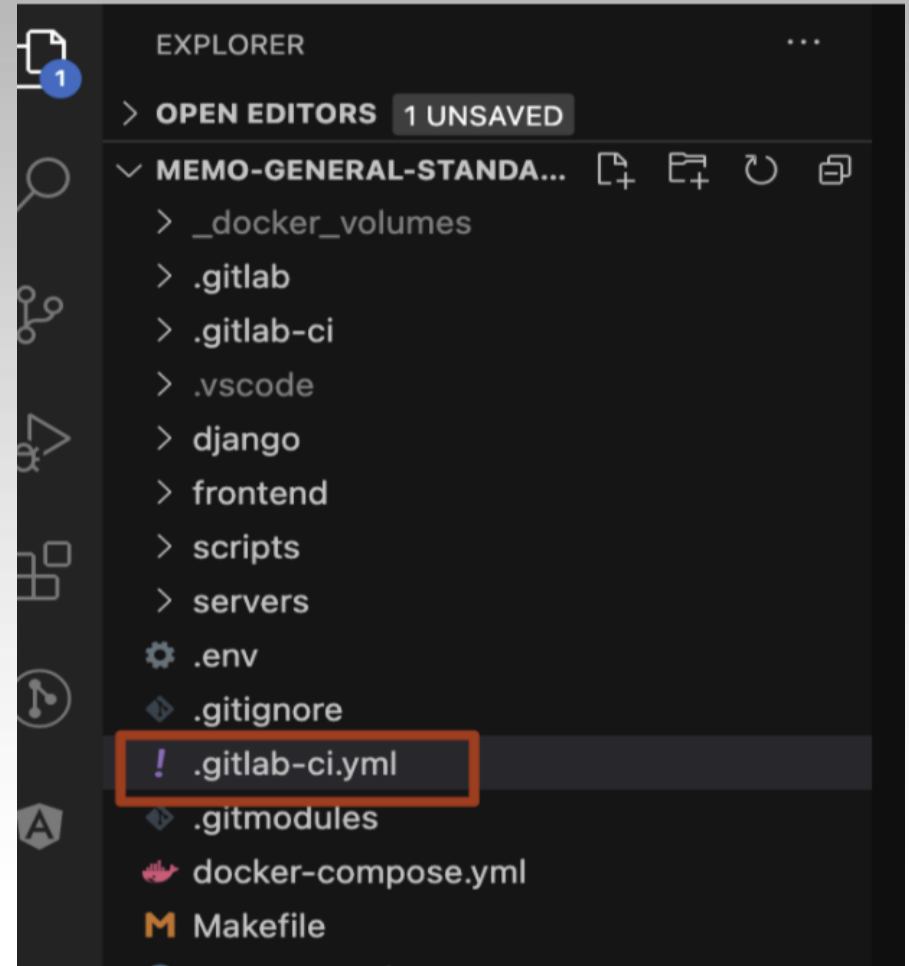


ตัวอย่างการทำงานแบบ CI/CD ด้วย Gitlab

ขั้นตอนการติดตั้งเพื่อเริ่มใช้งาน

Gitlab CI/CD

1. เพิ่มไฟล์ชื่อ “gitlab-ci.yml”
ไปยัง Root ของ Project โดยชื่อ
ไฟล์นี้เป็นมาตรฐานที่ตั้งโดย
Gitlab





ตัวอย่างการทำงานแบบ CI/CD ด้วย (2)

ในไฟล์จะต้องมีการกำหนด Stage หรือ Topic of Deployment ต่างๆ ซึ่งผู้ใช้งานสามารถออกแบบและกำหนดเองได้ตามความเหมาะสม

ภายใต้ Stage ผู้ใช้งานสามารถกำหนด Job ได้ว่า ในแต่ละ Stage มี Job อะไรบ้าง และใน Job นั้นเองจะมี Script เป็นตัวกำหนดว่าต้องทำการ Run Command ใดบ้าง

โปรแกรมเมอร์สามารถสร้าง เงื่อนไข (Condition) สำหรับการรัน Job ได้ โดยหนดว่าให้ Run ทุกครั้งเมื่อมีการทำ Merger Request (MR) หรือ มีการ Push Changes ไปยังไฟล์ที่กำหนด ดังนั้นหากเรามีการติดตั้งเครื่องมือเหล่านี้ครบถ้วน เราจะสามารถใช้ Automated Process ในการทำผ่านกระบวนการ CI/CD ได้



ตัวอย่างการทำงานแบบ CI/CD ด้วย (3)

```
stages:  
  - build-image  
  - unit-test  
  - build-dev  
  - deploy-dev
```

```
build-backend-for-test:  
  stage: build-image  
  image: docker:stable  
  services:  
    - docker:dind
```

if you want to build docker image, use this code

```
script:  
  - docker login -u $DOCKER_HUB_USERNAME -p  
    $DOCKER_HUB_PASSWORD  
  - docker build --tag dev ./django  
  - docker push c0d1um/support_portal:django-dev
```

setup command to build image here.

```
only:  
  refs:  
    - master  
    - merge_requests  
  changes:  
    - django/dockerfile  
    - django/requirements.txt
```

setup condition when to run this job.

Build-image

✓ build-backend... ↺

✓ build-frontend... ↺



ตัวอย่างการทำงานแบบ CI/CD ด้วย (4)

2. ติดตั้ง Application หรือเครื่องมือที่จะทำการเชื่อมต่อ Gitlab CI/CD กับ Gitlab Repository เพื่อสร้าง Automated Process สำหรับการ deploy ทุกๆ ครั้งที่มีการ Push Commit เข้ามายัง Gitlab

หนึ่งในเครื่องมือที่เราใช้อยู่คือ Gitlab Runner ซึ่งเป็นเครื่องมือที่ติดตั้งง่าย เหมาะกับการใช้งาน และสามารถจัดการการทำงานได้ตามความต้องการ ช่วยสนับสนุนการทำงานของ Gitlab CI/CD

โดยเครื่องมือนี้เป็น Open Source ของ Gitlab ที่จะให้ Gitlab CI/CD สามารถทำงานได้ หรือช่วยในการ Run Job ตามที่ถูกกำหนดไว้ในไฟล์ gitlab-ci.yml



ตัวอย่างการทำงานแบบ CI/CD ด้วย (5)

หากมีการติดตั้ง Gitlab Runner พร้อมกับเพิ่มไฟล์ gitlab-ci.yml แล้ว ทุกๆครั้งที่ Push Code/Commit ตามเงื่อนไขที่กำหนด ระบบจะเริ่มกระบวนการ CI/CD ทันที

โดยเมื่อมีการ Push Code ไปยังไฟล์ที่กำหนด ระบบจะเริ่มกระบวนการ CI/CD เพื่อทำการ Deploy ผู้ใช้งานจะสามารถตรวจสอบกระบวนการนี้ได้ที่เมนู “CI/CD” จากนั้นให้เข้าไปยัง Pipeline

เมื่อเข้ามายังเมนูดังกล่าวจะเห็น Pipeline ของการ Run Command ต่างๆ เพื่อการ Deploy หากอธิบายง่ายๆ Pipeline จะช่วยให้เรามองเห็นรูปแบบการทำงานของ CI/CD ใน Stage ต่างๆ เพื่อตรวจสอบข้อมูลการทำงานและสถานะ รวมถึงสามารถกดเข้าไปยังรายการนั้นๆได้



ตัวอย่างการทำงานแบบ CI/CD ด้วย (6)

 Codium Project

Project overview

Repository

Issues 57

Merge requests 0

CI/CD

Pipelines

Jobs

Schedules

Operations

Packages & Registries

Analytics

Snippets

Members

Collapse sidebar

CODIUM > Major > Codium Project > Pipelines

All 1,000+ Finished Branches Tags

Filter pipelines

Status	Pipeline	Triggerer	Commit	Stages	Duration
passed	#316589579 latest		main -> 7a98fb17 Merge branch '1424-est...	       	00:17:33 3 hours ago
passed	#316561649		main -> 794ccc71 Merge branch '1359-lice...	       	00:17:00 4 hours ago
passed	#316557070 latest detached		1649 -> cb763db2 Change from update rea...	    	00:12:37 5 hours ago
passed	#316147382 latest detached		1648 -> dbffb7ce Fix imports	    	00:13:22 19 hours ago
passed	#315998172		main -> 14df58dd Merge branch '1428-fix-...	     	00:13:13 22 hours ago



ตัวอย่างการทำงานแบบ CI/CD ด้วย (7)

เมื่อกดเข้าไปผู้ใช้งานจะเห็น Pipeline ของรายการนั้นๆ โดยให้สังเกตว่า หัวข้อหลักใน Pipeline เช่น Unit-test, Build-dev และ Deploy-dev คือ Stages ตามที่เราตั้งค่าไว้ในไฟล์ gitlab-ci.yml ดังรูป

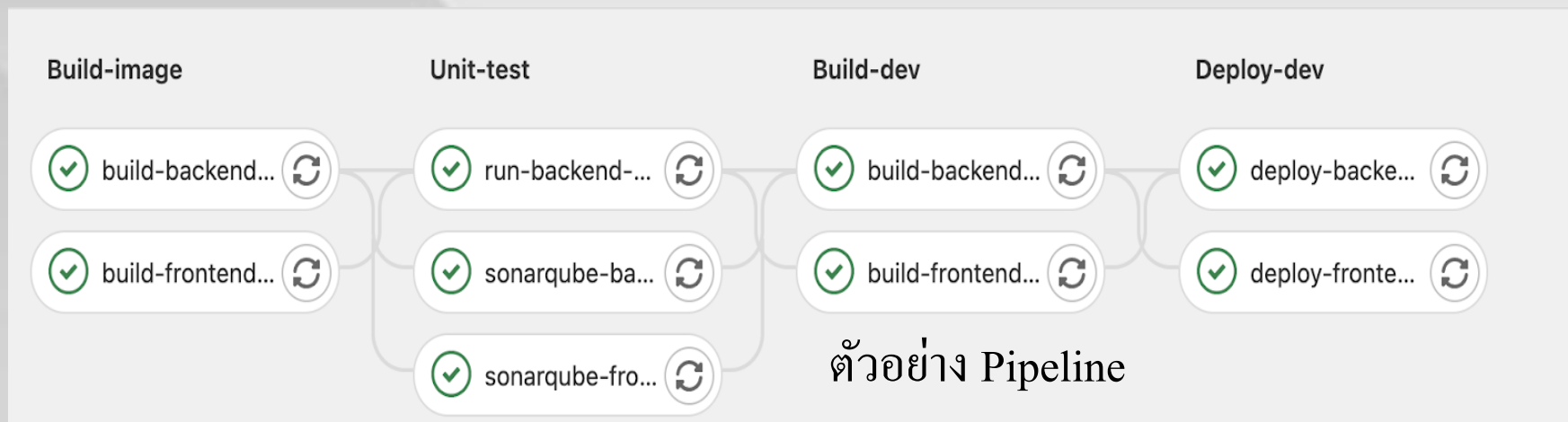
```
! .gitlab-ci.yml • api_urls.py  
  
! .gitlab-ci.yml  
1  stages:  
2    - build-image  
3    - unit-test  
4    - build-dev  
5    - deploy-dev  
6
```



ตัวอย่างการทำงานแบบ CI/CD ด้วย (8)

```
! .gitlab-ci.yml • api_urls.py  
! .gitlab-ci.yml  
1  stages:  
2    - build-image  
3    - unit-test  
4    - build-dev  
5    - deploy-dev  
6
```

ตัวอย่าง การตั้งค่า Stages ในไฟล์
gitlab-ci.yml





ตัวอย่างการทำงานแบบ CI/CD ด้วย (9)

ส่วนรายการทางด้านล่าง Stages คือ Job ที่เราตั้งค่าเอาไว้ เมื่อเกิดการ Deploy ระบบจะทำการ Trigger และเริ่ม Run Command ทั้งหมดตามที่กำหนดไว้ตามลำดับ จนกระทั่ง Code ถูก Deploy ไปยัง Server ที่กำหนดไว้ จะเห็นได้ว่า Process นี้มีการทำงานของคนที่เพียงแค่ขั้นตอนการ Push/ MR จากนั้นระบบที่ตั้งค่าไว้ จะดำเนินการ Deploy ทุกอย่างตามรูปแบบที่กำหนด จึงช่วยลดการทำงานที่ซ้ำซ้อน และช่วยให้โปรแกรมเมอร์สามารถทำงานอื่นๆ ได้อย่างมีประสิทธิภาพมากขึ้น

ทั้งหมดนี้คือข้อมูลเบื้องต้นสำหรับการทำ Gitlab CI/CD จริงๆแล้วยังมีเครื่องมืออื่นๆอีกมาก ที่ช่วยให้การทำงานเป็นไปได้ง่ายขึ้น



การทำงานแบบ Silo

ทุกวันนี้ ทั้ง ๆ ที่มีเครื่องมือดิจิทัลหรือซอฟต์แวร์ใหม่ ๆ ที่ออกแบบเพื่อการทำงานร่วมกันมากมายอย่างไม่เคยเกิดขึ้นมาก่อน แต่ผลสำรวจจาก Customer.com ได้สะท้อนว่า 40% ของพนักงานในบริษัท ไม่ได้รับการสนับสนุนที่เพียงพอจากเพื่อนร่วมงาน ด้วยเหตุผลว่า เป็นเรื่องของแผนก

นั่นหมายถึงองค์กรของคุณกำลังเข้าสู่ภาวะไซโล (Silo) กลายเป็น Siloed Organization หรือ Siloed Company เมื่อแผนกต่าง ๆ ในองค์กรเดียวกัน ไม่ยอมแลกเปลี่ยนข้อมูลหรือองค์ความรู้ระหว่างกัน



การทำงานแบบ Silo (2)

สัญญาณที่บ่งชี้ว่า องค์กรของคุณกำลังเข้าสู่ภาวะไซโล มี 5 ข้อ ดังนี้

1. ทำลายประสิทธิภาพการณ้ของลูกค้า เช่น บริษัทอีคอมเมิร์ซที่ไม่สามารถใช้เทคโนโลยีหรือช่องทางการสื่อสารเพื่อแบ่งแยกลูกค้าที่คิดจะซื้อของของคุณ กับลูกค้าที่ซื้อของไปแล้ว โดยทีมการตลาดดูแลลูกค้าที่ซื้อของไปแล้ว ราวกับคนที่ยังไม่ได้ซื้อของ เพราะฝ่ายเซลล์กับฝ่ายการตลาดไม่คุยกัน



การทำงานแบบ Silo (3)

2. ความแปลกหน้าในองค์กร สัญญาณเตือนภัยคือ หากคุณไม่รู้จักคนส่วนใหญ่ที่ทำงานนอกทีม หรือสิ่งที่พวกเขาทำในองค์กร ซึ่งไม่ได้หมายถึงให้คุณคุ้นเคยกับทุกคนในองค์กร แต่อย่างน้อย ถ้าเป็นบริษัทเล็ก ๆ ก็ควรรู้จักชื่อของทุกคน แต่ถ้าเป็นองค์กรใหญ่อย่างน้อยก็ควรจะรู้ชื่อและช่องทางติดต่อระดับบริหารแต่ละแผนก เพื่อให้เป็นวัฒนธรรมองค์กรที่ปัดเป่าไซโลได้



การทำงานแบบ Silo (4)

3. ภาวะ "เรา" กับ "เขา" ระหว่างแผนก จะทำให้เกิดภาวะการแข่งขัน และเป็นอุปสรรคต่อความสำเร็จขององค์กร เพราะไม่มีการแชร์ข้อมูลและขาดความร่วมมือร่วมใจ จากการกลัวว่าอีกทีมจะได้หน้า แต่ทีมคุณคือแพ้



การทำงานแบบ Silo (5)

4. พนักงานที่ถูกเพิกเฉย ซึ่งมีความเชื่อมโยงระหว่างไซโลกับการไม่
มีงาน คนที่ไม่ได้รู้สึกเป็นส่วนหนึ่งของทีม หรือถูกปฏิบัติแบบไม่
เหมือนคนอื่น อาจจะไม่มีความสุข รู้สึกไม่มีประโยชน์ และเสี่ยงต่อ
การแซร์หรือป่วนหัวสิ่งที่ไม่ดีกับเพื่อนร่วมงานคนอื่น ๆ



การทำงานแบบ Silo (6)

5. การทำงานซ้ำซ้อน เพราะไม่มีการสื่อสารกัน ไม่มีทางที่จะรู้ว่าสิ่งที่เราทำไป ได้ทำซ้ำกับคนอื่นหรือแผนกอื่นหรือไม่ ธุรกิจที่ขาดความร่วมมือจะมีคนและทีมงานที่ทำงานในโครงการที่คล้ายกัน ซึ่งนำไปสู่ความไร้ประสิทธิภาพและสูญเสียประสิทธิภาพการทำงานไปอย่างน่าเสียดาย

ดังนั้นไม่ว่าจะเป็นผู้บริหารหรือพนักงาน หากเจอสัญญาณเหล่านี้ให้เริ่มใช้กลยุทธ์เพื่อทลายการทำงานแบบไซโล ที่เป็นอุปสรรคในการสื่อสารและการทำงานร่วมกัน



Digital transformation



Digital transformation

หลายคนอาจจะเคยได้ยินประโยคนี้จากผู้บริหารกันบ่อย ๆ ในช่วง 2-3 ปีที่ผ่านมา ไม่มีอะไรสำคัญไปกว่าการทำ Digital transformation ให้กับธุรกิจของเรา ที่มาของประโยคนี้ค่อนข้างชัดเจน เนื่องจากเทคโนโลยีได้สร้างรูปแบบใหม่ของธุรกิจในการส่งมอบคุณค่าให้ลูกค้า เปลี่ยนแปลงรูปแบบของการทำธุรกิจ และเปลี่ยนแปลงตลาดที่มีอยู่ในปัจจุบัน



Digital transformation (2)

การเปลี่ยนแปลงของเทคโนโลยีไม่ได้ถือว่าเป็นเรื่องใหม่ แต่ในรอบของการเปลี่ยนแปลงนี้เป็นการเปลี่ยนแปลงในจังหวะที่เร็วขึ้นกว่าเดิม ผู้บริหารบางท่านได้กล่าวไว้ว่าความเสี่ยงที่จะถูก Disrupt เป็นตัวกระตุ้นสำคัญที่ทำให้องค์กรต้องมาทำ Digital Transformation ในขณะที่เทคโนโลยีใหม่ๆ ที่เกิดขึ้นอาจจะส่งผลกระทบต่อธุรกิจในยุคเดิม แต่ในขณะเดียวกันก็สร้างโอกาสใหม่ๆ ที่ไม่เคยมีมาก่อนได้เช่นกัน



Digital transformation (3)

Digital Transformation คืออะไร





Digital transformation (4)

Digital Transformation คือการนำเอาเทคโนโลยีดิจิทัลมาประยุกต์ใช้กับทุกฟังก์ชันในการทำธุรกิจ ตั้งแต่รูปแบบในการทำธุรกิจ (Business model) จนถึงการสร้างประสบการณ์ของลูกค้า จนถึงกระบวนการทำงานภายในองค์กร โดยพื้นฐานแล้ว Digital transformation เป็นการเปลี่ยนวิธีในการทำงาน และการส่งมอบคุณค่าให้กับลูกค้า โดยรวมถึงเกี่ยวข้องกับการเปลี่ยนวัฒนธรรมในการทำงานที่ทำให้องค์กรต้องคิดค้นหาวิธีการทำงานให้ดีขึ้นอยู่เสมอ พร้อมทดลองสิ่งใหม่ๆ และคุ้นเคยกับความล้มเหลว โดยมีจุดประสงค์เพื่อยกระดับความคาดหวังของลูกค้า และสร้างโอกาสใหม่ๆ ให้กับธุรกิจนั่นเอง



Digital transformation (5)

โดยเทคโนโลยีที่นำมาใช้มีตั้งแต่เทคโนโลยี AI การทำงานแบบอัตโนมัติ (Automation) เทคโนโลยี Cloud แบบลูกผสม (Hybrid Cloud) รวมถึงเทคโนโลยีอื่นๆเพื่อให้สามารถใช้ประโยชน์สูงสุดจากข้อมูลที่มีอยู่ได้ และนำมาสร้างเป็นกระบวนการทำงานแบบอัจฉริยะ (Intelligent workflow) ส่งผลให้องค์กรสามารถตัดสินใจได้อย่างรวดเร็ว และตรงจุดมากยิ่งขึ้น และยังตอบสนองต่อการเปลี่ยนแปลงในตลาดได้อย่างรวดเร็วอีกด้วย



Digital transformation (6)

4 ประเภทของการทำ Digital Transformation

1. Process Transformation

กิจกรรมที่องค์กรให้ความสำคัญมากที่สุด คือกระบวนการในการทำงานทางธุรกิจ ข้อมูล, การวิเคราะห์ข้อมูล, การใช้ API, Machine Learning รวมถึงเทคโนโลยีอื่นๆเปิดโอกาสให้ธุรกิจสามารถสร้างสรรค์กระบวนการทำงานใหม่ๆได้ง่ายมากยิ่งขึ้น โดยมีวัตถุประสงค์เพื่อลดต้นทุน ลดรอบของการทำงาน และเพิ่มคุณภาพ และสามารถประยุกต์ใช้ได้กับธุรกิจหลากหลายประเภท



Digital transformation (7)

ตัวอย่างเช่น Domino's Pizza



แหล่งที่มา <https://anyware.dominos.com/>



Digital transformation (9)

ตัวอย่างเช่น Domino's Pizza บริษัทพิซซ่าชื่อดังจากประเทศสหรัฐอเมริกาได้นำกระบวนการ Process Transformation มาใช้ในการยกระดับการให้บริการลูกค้า โดยสร้างกระบวนการในการสั่งพิซซ่าใหม่ ลูกค้าสามารถสั่ง Domino's Pizza จากอุปกรณ์(Device)ได้ทุกรูปแบบที่มีชื่อว่า Dominos' AnyWare ไม่ว่าจะเป็น Google home, Alexa, Slack, Facebook messenger, ข้อความ, Twitter, สมาร์ททีวี, Smartwatch เป็นต้น และด้วยวิธีการสั่งในรูปแบบใหม่ทำให้ยอดขาย Domino's Pizza พุ่งแซง ยอดขายของ Pizza hut ไปได้นอกจากนั้นหลายบริษัทได้มีการนำระบบการทำงานอัตโนมัติแบบหุ่นยนต์ (Robotic Process Automation: RPA) เข้ามาใช้ในการทำงานหลังบ้าน เช่น งานทางด้านบัญชี และกฎหมาย เป็นต้น Process Transformation สามารถสร้างมูลค่าได้อย่างมีนัยสำคัญให้กับธุรกิจ และการนำเทคโนโลยีมาใช้ขั้นตอนนี้สามารถทำได้อย่างรวดเร็ว และถือเป็นพื้นฐานที่สำคัญสำหรับการทำ Digital Transformation



Digital transformation (10)

2. Business Model Transformation

บางธุรกิจนำเทคโนโลยีดิจิทัลมาใช้ในการเปลี่ยนแปลงรูปแบบในการทำธุรกิจ ในขณะที่ Process Transformation โฟกัสไปที่ขอบเขตบางอย่างในธุรกิจ แต่ในการทำ Business Model Transformation มุ่งไปที่การสร้างพื้นฐานธุรกิจในการส่งมอบคุณค่าให้กับลูกค้าในอุตสาหกรรมนั้นๆ ตัวอย่างที่เห็นได้ชัด ได้แก่ Netflix ที่เข้ามาเปลี่ยนรูปแบบของการจำหน่ายวิดีโอ Apple เปลี่ยนรูปแบบของการจำหน่ายเพลงโดยสร้าง iTunes ขึ้นมา Uber เปลี่ยนรูปแบบอุตสาหกรรมแท็กซี่ การทำ Business Model Transformation สามารถเกิดขึ้นได้ในทุกอุตสาหกรรม เช่น ในธุรกิจประกันรถยนต์ บริษัทประกันภัย Allstate และ Metromile ได้ใช้ข้อมูลที่มีอยู่ และนำมาวิเคราะห์เพื่อหา insight ลูกค้า จากข้อมูลที่ได้จึงทำให้ทั้ง 2 บริษัทพบว่าควรขายประกันภัยรถยนต์แบบแยก และคิดเบี้ยประกันภัยรถยนต์ตามจำนวนกิโลเมตรที่ใช้ไปจริง จึงทำให้เกิดการเปลี่ยนแปลงในวงการประกันรถยนต์



Digital transformation (11)

ความซับซ้อน และ โอกาสทางกลยุทธ์เหล่านี้ต้องใช้ความร่วมมือร่วม และ Leadership จากทางทีมกลยุทธ์ และหน่วยงานที่เกี่ยวข้อง และมักจะตั้งเป็น หน่วยงานพิเศษขึ้นมาเพื่อคิดค้นรูปแบบทางธุรกิจใหม่ ในขณะที่เดียวกันก็ยัง ดำเนินธุรกิจเดิมไปพร้อมกัน ในการเปลี่ยนแปลงรูปแบบการทำธุรกิจในการ ส่งมอบคุณค่าให้ลูกค้า องค์กรที่ประสบความสำเร็จในการเปลี่ยนแปลง รูปแบบในการดำเนินธุรกิจมักจะมีโอกาสเติบโตอย่างมีนัยสำคัญ และองค์กร ควรให้ความสำคัญกับเรื่องนี้



Digital transformation (12)

ตัวอย่างเช่น Netflix



Digital transformation (13)

3.Domain Transformation

เป็นข้อที่องค์กรส่วนใหญ่ให้ความสำคัญน้อย แต่สามารถสร้างโอกาสทางธุรกิจที่สูงมาก คือการทำ Domain Transformation เทคโนโลยีใหม่ๆ ได้กำหนดค่านิยมใหม่ให้กับสินค้าและบริการ ทำให้เส้นแบ่งระหว่างแต่ละอุตสาหกรรมบางลง และสร้างกลุ่มของกลุ่มแข่งขันใหม่ขึ้นมา สิ่งสำหรับผู้บริหารส่วนใหญ่ไม่ค่อยได้คำนึงถึง คือ โอกาสที่แท้จริงที่เทคโนโลยีใหม่ๆ เหล่านี้จะเข้ามาช่วยปลดล็อกให้เข้าสู่ธุรกิจใหม่ได้ง่ายขึ้น และ Domain Transformation มักจะเป็นโอกาสให้ธุรกิจได้สร้างคุณค่าใหม่ๆ ให้กับลูกค้า



Digital transformation (14)

ตัวอย่างทางธุรกิจที่เห็นได้ชัดจากการนำ Domain Transformation มาใช้ คือ Amazon





Digital transformation (15)

ตัวอย่างทางธุรกิจที่เห็นได้ชัดจากการนำ Domain Transformation มาใช้ คือ Amazon เว็บไซต์ขายของออนไลน์เบอร์ 1 จากประเทศสหรัฐอเมริกา Amazon ได้ขยายธุรกิจไปยังตลาดใหม่ด้วยการเปิดตัว Amazon Web Service (AWS) ซึ่งในปัจจุบันเป็นบริษัทผู้ให้บริการด้าน Cloud computing ที่ใหญ่ที่สุด โดยก่อนหน้านี้ตลาดผู้ให้บริการ Cloud computing จะเป็นตลาดของผู้ให้บริการ IT รายใหญ่ของโลก ได้แก่ IBM และ Microsoft เหตุผลที่ทำให้ Amazon สามารถเข้าสู่ตลาด Cloud computing ได้ คือความสามารถทางด้านเทคโนโลยีดิจิทัลอย่างแข็งแกร่งที่ Amazon ได้สร้างไว้ ทั้งในเรื่องของพื้นที่การจัดเก็บข้อมูล รวมถึง Computing database ที่สนับสนุนธุรกิจค้าปลีกซึ่งเป็นธุรกิจหลักของทาง Amazon ประกอบกับมีความสัมพันธ์อันดีกับธุรกิจใหม่ๆที่กำลังเติบโตเป็นจำนวนมากที่ต้องการใช้บริการด้านนี้ AWS ไม่ได้เป็นเพียงการขยายไปสู่ธุรกิจใหม่ของทาง Amazon แต่เป็นธุรกิจที่แตกต่างออกไปจากเดิมในอีกตลาดการแข่งขันอย่างชัดเจน และธุรกิจ AWS ในตอนนี้มีกำไรคิดเป็น 60% ของกำไรของ Amazon ทั้งหมด



Digital transformation (15)

เป็นเรื่องที่น่าดึงดูดใจของผู้บริหารองค์กรธุรกิจ Non-tech ที่มองประสบการณ์ของ Amazon และบริษัทเทคโนโลยีอื่นๆ เช่น Apple หรือ Google เป็นเรื่องที่พิเศษ เนื่องจากความสามารถในการเข้าถึง และนำเทคโนโลยีมาใช้ให้เกิดประโยชน์สูงสุดอาจจะดีกว่าบริษัทอื่นๆ แต่โลกดิจิทัลในยุคปัจจุบัน ช่องว่างเรื่องเทคโนโลยีไม่ได้เป็นเรื่องสำคัญอีกต่อไป เนื่องจากบริษัทสามารถเข้าถึงเทคโนโลยีเพื่อทำให้ธุรกิจเติบโตได้ในต้นทุนที่ไม่แพง และมีประสิทธิภาพมากยิ่งขึ้น การสร้างพื้นฐานทางเทคโนโลยีจะช่วยปลดล็อกการขยายตัวของธุรกิจ (เทคโนโลยี AI, Machine Learning, internet of things (IOT), augmented reality และอื่นๆ) สามารถหาได้ทั้งจากบริษัทผู้ให้บริการ IT ขนาดใหญ่ เช่น Microsoft , IBM รวมไปถึงบริษัท Start-up ที่กำลังเติบโต ซึ่งเป็นที่ที่มี Innovation เกิดขึ้นมากมาย องค์กรที่รู้วิธีการเข้าถึงเทคโนโลยีเหล่านี้ และใช้เทคโนโลยีให้เกิดประโยชน์สูงสุดจะเป็นองค์กรที่สามารถเติบโตได้อย่างไม่หยุดยั้ง



Digital transformation (16)

4.Cultural/Organizational Transformation

การทำ Digital Transformation ในระยะยาวนั้นต้องมีปรับเรื่อง mindset ขององค์กร กระบวนการในการทำงาน การบริหารทรัพยากรบุคคล และความสามารถในโลกดิจิทัล กันใหม่ องค์กรระดับโลกที่ให้ความสำคัญเรื่องดิจิทัลต้องมีเรื่องของกระบวนการทำงาน แบบ Agile ทดสอบและเรียนรู้ ให้อำนาจในการตัดสินใจ มีระบบ Ecosystem ที่แข็งแกร่ง และใช้ขั้นตอนเหล่านี้ในการเปลี่ยนแปลงองค์กร



Digital transformation (17)

ตัวอย่างเช่นบริษัท Experian ผู้ให้บริการด้านการเช็กเครดิตทางการเงินส่วนบุคคลจากประเทศไอร์แลนด์ เป็นหนึ่งในบริษัทที่ประสบความสำเร็จในการทำ Digital Transformation มากที่สุดแห่งหนึ่ง ได้เปลี่ยนแปลงองค์กร โดยการนำวิธีการทำแบบ Agile มาใช้ และเพิ่มความร่วมมือในการทำงานในกระบวนการทำงาน และผลักดันให้เกิดความสนใจในเรื่องของข้อมูล และความเป็นไป ของบริษัทในมุมที่กว้างขึ้น เช่นเดียวกันกับบริษัท Pitney Bowes บริษัทผู้ให้บริการเกี่ยวกับไปรษณีย์มากกว่า 100 ปี ประสบความสำเร็จในการเปลี่ยนตัวเองเป็น Tech Company โดยการผลักดัน วัฒนธรรมการสร้างสิ่งใหม่ๆ (Innovation) และเปลี่ยนค่านิยมขององค์กร โดยมุ่งเน้นที่ การให้ลูกค้าเป็นศูนย์กลาง (Customer-centricity)



Digital transformation (18)

บริษัท Experian

The screenshot shows the Experian website interface. At the top, there is a navigation bar with links for Consumer, Small Business, Business, Data & Marketing, and Your privacy. On the right, it says 'United Kingdom' and 'Global Sites'. Below the navigation bar, the Experian logo is followed by links for 'Credit score & report', 'Compare credit offers', 'Identity theft', and 'Help & assistance', each with a dropdown arrow. A 'Log in' button is on the far right. The main content area features a large graphic of a person celebrating a credit score of 973, with a message 'You've boosted your score +24'. To the left, there is a section titled 'Get your FREE Experian Credit Score' with the text 'Join 12 million others who already have the UK's most trusted credit score' and a 'Sign up free' button. At the bottom, there is a link to 'Learn more' about the Statutory Credit Report.

Consumer Small Business Business Data & Marketing Your privacy United Kingdom Global Sites

experian. Credit score & report ▾ Compare credit offers ▾ Identity theft ▾ Help & assistance ▾ Log in

Get your FREE Experian Credit Score

Join 12 million others who already have the UK's most trusted credit score

Sign up free

973

You've boosted your score +24

You can also get a free copy of your [Statutory Credit Report](#), which shows your credit history. [Learn more ▾](#)



Digital transformation (19)

สรุป Digital transformation

เมื่อเทคโนโลยีมีการเปลี่ยนแปลงมากขึ้น ธุรกิจในแต่ละอุตสาหกรรม จะถูกบังคับให้มีการเปลี่ยนแปลงอย่างต่อเนื่ององค์กรที่ใส่ใจ และนำเรื่อง Digital transformation ไปปรับใช้ในหลากหลายมิติ

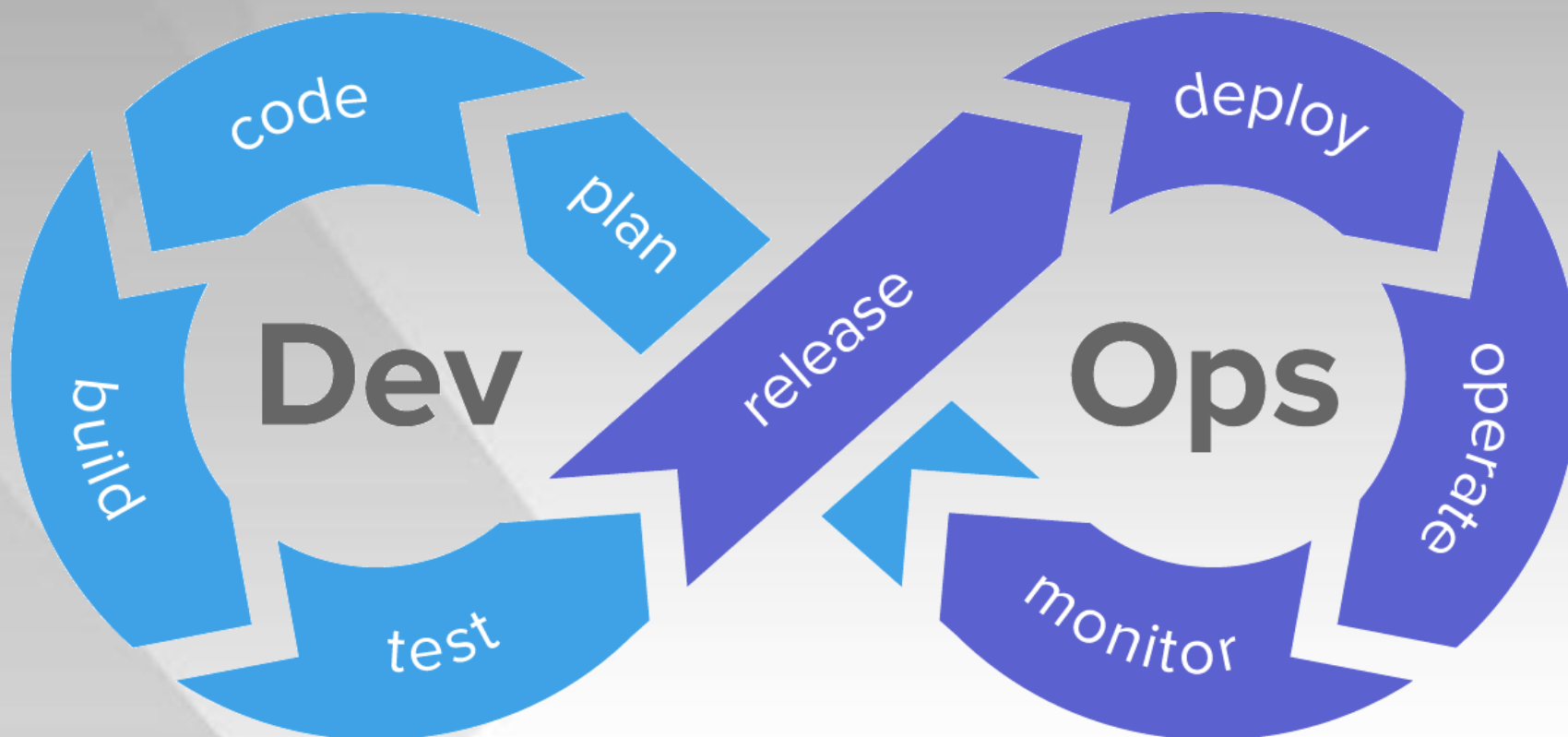
องค์กรนั้นจะมีโอกาสที่จะประสบความสำเร็จมากกว่าองค์กรที่ไม่ได้มี
คำนึงถึงเรื่อง Digital transformation นี้เลย



Digital transformation

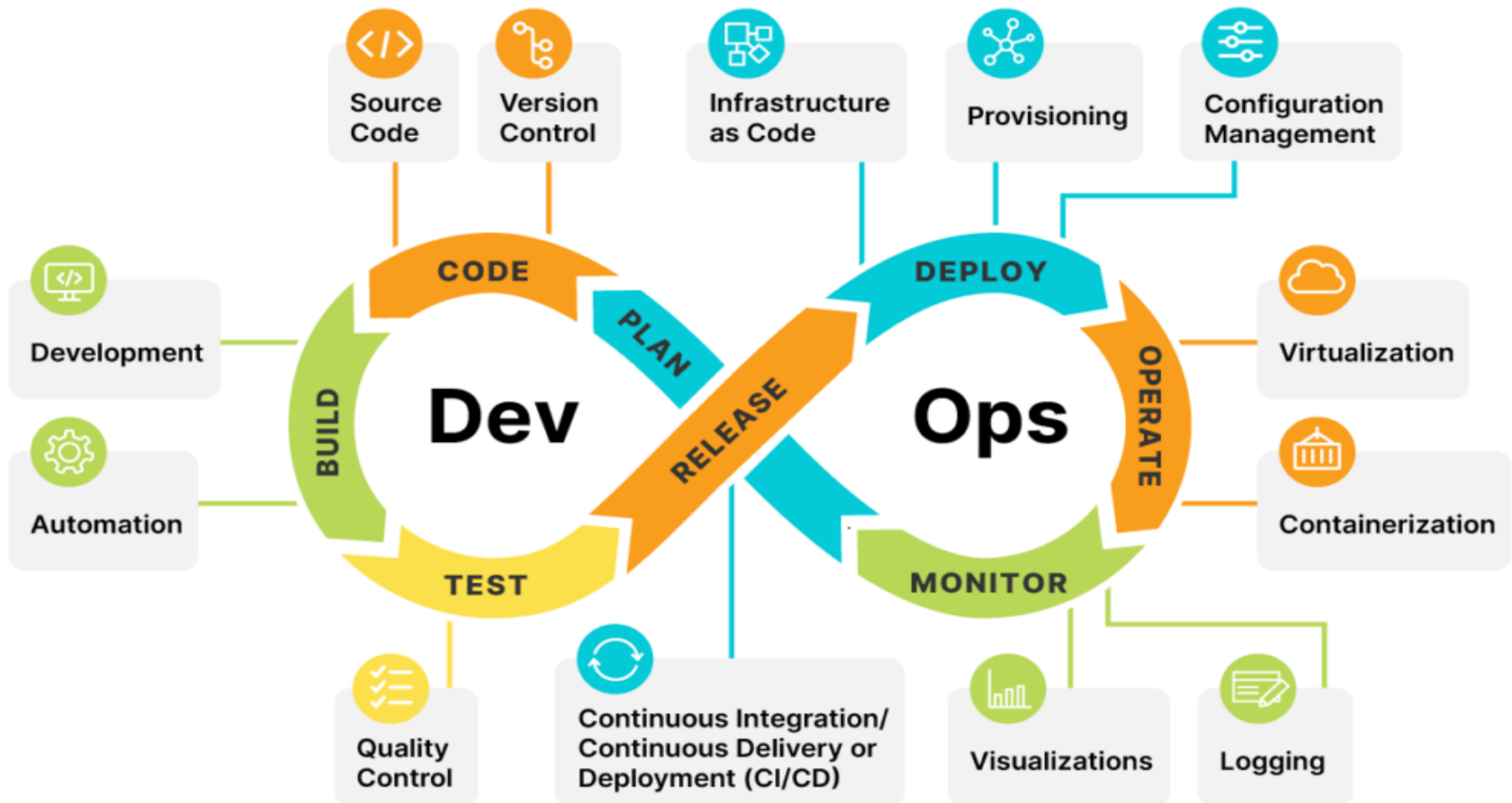


กระบวนการแบบ DevOps



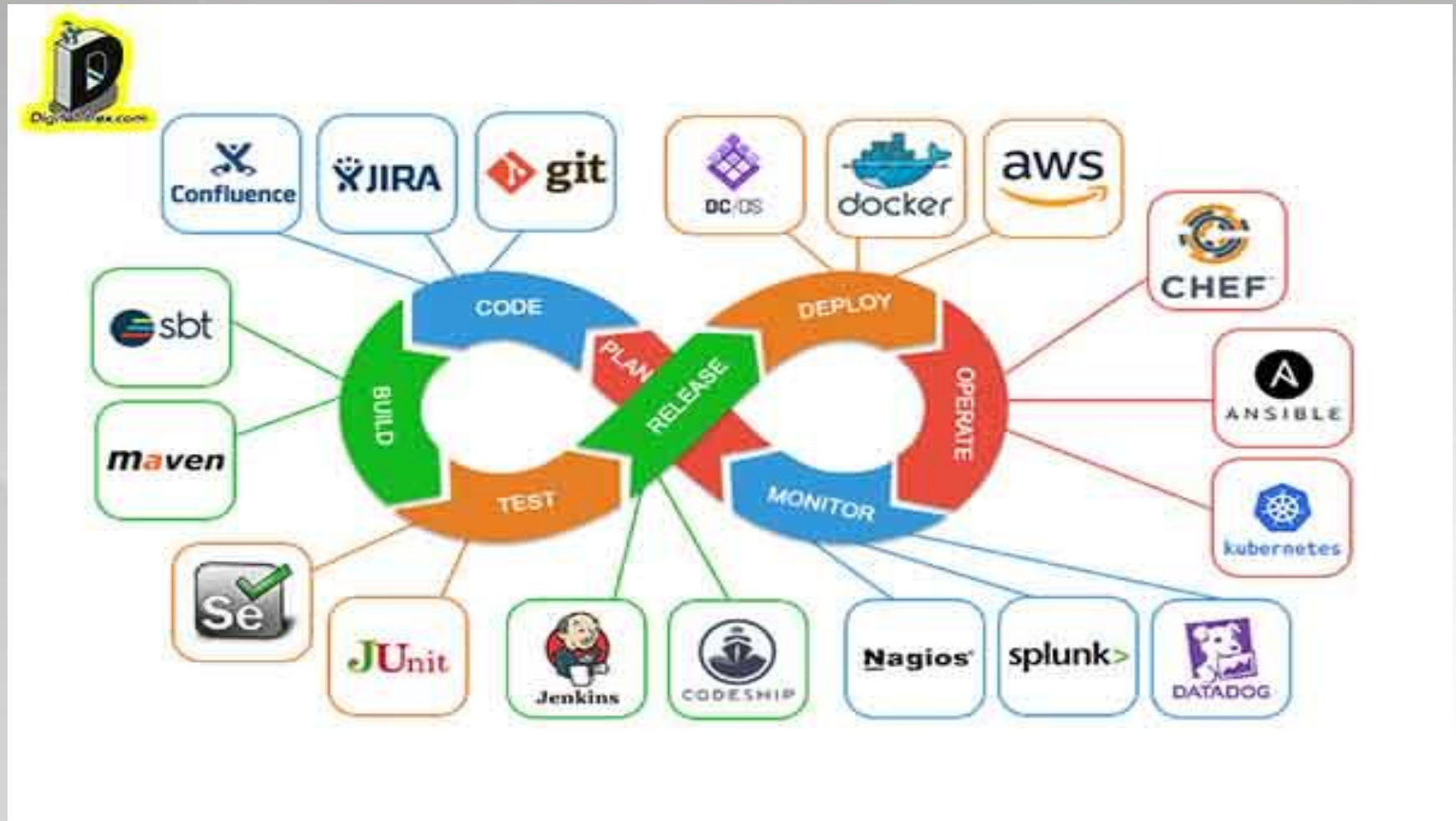


กระบวนการแบบ DevOps (2)





กระบวนการแบบ DevOps (3)





กระบวนการแบบ DevOps (4)

DevOps คือแนวคิดในการประสานการทำงานในฝ่ายพัฒนา (Development) เข้ากับฝ่ายดำเนินการ (Operational) ให้มีความลื่นไหลตลอด Life Cycle แต่จริงๆ แล้วก็ไม่ได้ถือว่าเป็นอะไรที่ใหม่มากเพราะก็มีมาประมาณตั้งแต่ช่วง 2010 แล้วสำหรับในตลาด IT เมืองนอก (ตั้งแต่ปี 2012) เพียงแต่ว่าในประเทศไทยได้รับความนิยมมากขึ้นเพราะเป็นช่วงที่หลาย ๆ องค์กรทั้ง Startup และ Enterprise ใหญ่ๆ กำลังเข้าสู่ Digital Transformation กันอย่างเต็มตัวไม่ว่าจะเป็นอุตสาหกรรมธนาคารอย่าง SCB ที่เริ่มมีการเปิด Open API ให้ Third Party เข้ามาต่อ API การชำระเงินการธนาคารได้ โดยไม่ได้เป็นระบบปิดอีกต่อไปหรือ Startup ซึ่งมีการปล่อย Features ใหม่ๆ ออกมาเพื่อตอบโจทย์ความต้องการของผู้ใช้งาน ที่ต้องการอะไรรวดเร็วมี Feedback Loop ไวๆ



กระบวนการแบบ DevOps (5)

ถ้าเราจะกลับมามองที่ว่า DevOps นั้นเกี่ยวข้องกับ Digital Transformation ภายในองค์กรบ้าง ก็ต้องลองย้อนกลับไปทีในมุมมองของธุรกิจที่ขับเคลื่อนด้วยระบบ IT เป็นหลักอย่างธนาคารที่ต้องมีระบบบันทึก Transaction การเงินลง Database เชื่อมต่อชำระเงินผ่านระบบ Online ที่มีจำนวนผู้ใช้ Concurrent พร้อมกันมาก ๆ การปล่อยให้ระบบล่มนั้นถือเป็นสิ่งที่ยอมไม่ได้เลย และในขณะเดียวกันก็ต้องมีการนำ Features ใหม่ขึ้นๆในระบบ เพื่อตอบสนอง Business Model ขององค์กรเช่นเดียวกันด้วย แล้วจะอย่างไรให้ Enterprise ใหญ่ ๆ เหล่านี้สามารถ Delivery Features ใหม่ๆได้อย่างรวดเร็วในขณะเดียวกันก็มีเรื่องของ Operational ที่ Available ตลอดเวลาให้สอดคล้องกับ Service Level Agreement ด้วยเช่นกัน ?



กระบวนการแบบ DevOps (6)

เราต้องมาทำการเข้าใจปัญหาที่อยู่ในองค์กรเราก่อนว่าปัจจุบันนี้การทำงาน
ของ Enterprise ใหญ่ๆในบางที่ที่กำลังอยู่ในช่วง Transform ตัวเองก็สามารถ
เข้าไปแข่งกับยุค Digital ซึ่งต้องใช้ IT และความรวดเร็วเป็นหลักมีปัญหา
อะไรบ้าง



กระบวนการแบบ DevOps (7)

สิ่งที่มักพบคือฝ่าย Development Team ซึ่งเป็นทีมสร้างซอฟต์แวร์หรือออก
ไอเดียใหม่ๆในการสร้าง Features ให้กับองค์กร ไม่ว่าจะเป็นการเขียน
ซอฟต์แวร์บนมือถือการสร้าง API หรือการแก้บั๊กการประมวลผล Analytics
ต่างๆนั้น ตัวชี้วัดความสำเร็จก็คือการ “สร้างสิ่งนั้นสำเร็จ” ยังมี Features มาก
เท่าไรก็ยังสะท้อนถึงประสิทธิภาพของทีม Development มากเท่านั้น แต่เมื่อ
เรากลับมาดูที่ Process ขององค์กรจะพบว่าต้องการเรื่องของการทำ Document
ที่ชัดเจน ซึ่งมีความซับซ้อนในการผ่านขั้นตอนการอนุมัติในแต่ละขั้นตอน
มากมาย กว่าจะสามารถนำซอฟต์แวร์ที่พัฒนาขึ้นมา ไปปล่อยใช้งานหรือ
Deploy ได้



กระบวนการแบบ DevOps (8)

ซึ่งเมื่อได้รับซอฟต์แวร์มาแล้วฝ่าย Operational ที่เป็นทีมดูแลเรื่องของ Infrastructure Network ในองค์กรก็จะต้องนำซอฟต์แวร์เหล่านี้ขึ้น Server และค่อยทำการ Deploy แบบ Manual ด้วยตนเอง ซึ่งตรงจุดนี้สังเกตว่าการที่ซอฟต์แวร์จะผ่านขั้นตอนเอกสารมาได้ ก็อาจจะใช้เวลาหลายสัปดาห์และกว่าจะถึงขั้นตอนของทีม Operational นำระบบขึ้น Server หรือไปขึ้น Cloud ก็ยังมีช่วงเวลาที่ต้องรอคอยอีกอาจจะเป็นขั้นต่ำ 2 อาทิตย์หรืออาจจะกลายเป็นหลักเดือนเลยก็เป็นไปได้ ซึ่งแน่นอนว่ากว่า Developer จะมีการขอ Deploy Feature ก็อาจจะเป็นทุกสามเดือนครั้ง หรือบางทีอาจจะสร้างมาปุ๊บแล้วก็ปล่อยให้มันรันยาว ๆ ไปเป็นปีไม่มีแก้บั๊กใด ๆ



กระบวนการแบบ DevOps (9)

ปัญหานั้นไม่ได้อยู่เพียงแค่การมี Features เพราะขั้นตอนการทำ Process แบบ Waterfall Traditional เพียงอย่างเดียวแต่ปัญหาจะมาก่ออยู่ในส่วนของ Technical ด้วยเช่นกันซึ่งการที่พัฒนาของเรา มีการเปลี่ยนแปลงครั้งละมาก ๆ ทำให้เกิดปัญหาเรื่องของ Compatibility ที่จะมี Breaking Change เยอะซึ่งอาจจะส่งผลไปยัง Infrastructure ที่เคยเตรียมไว้ด้วยเช่นกัน และทำให้ทีม Operational เองก็ไม่ทราบว่าจะตรวจสอบปัญหาได้จากที่ใดเพราะว่ามีการเปลี่ยนแปลงที่มากเกินไปในเมื่อถึงเวลา Upgrade Version ของซอฟต์แวร์ ดังนั้นการแก้ปัญหานั้นที่ควรจะเป็นคือการ อัปเดตซอฟต์แวร์ของเราทีละน้อย ๆ แต่บ่อยๆ เพราะเมื่อเราค่อยอัปเดตจะทำให้เราสามารถ Identify กลับไปยังต้นตอของปัญหาที่เกิดขึ้นได้ง่ายขึ้น



กระบวนการแบบ DevOps (10)

และยังทำให้เราเองสามารถทราบว่าการเปลี่ยนแปลงเหล่านั้นส่งผลกระทบต่อ Component ใด ๆ ในระบบบ้าง รวมทั้งทำให้เราสามารถ Mitigate ปัญหาเหล่านั้นได้ง่ายขึ้น เหมือนการปลูกต้นไม้ที่เราค่อย ๆ ปรับสูตรสารอาหารไปทีละส่วนให้เหมาะสม ย่อมดีกว่าการทำตามสูตรครั้งเดียวแล้วไม่สามารถสังเกตได้ว่าต้นไม้ที่เราปลูกนั้น เราให้แร่ธาตุหรือสารอาหารมากเกินไป น้อยไป หรือพอดีแล้ว



กระบวนการแบบ DevOps (11)

ผลของการปรับเปลี่ยนทีละน้อยยังทำให้ฝ่าย Operational สามารถเตรียม Infrastructure ได้อย่างเหมาะสมอีกด้วยว่า ซอฟต์แวร์ที่มีการเปลี่ยนแปลง ต้องการ Network เพิ่มหรือขยาย Hardware สำหรับ Computing เพิ่มมากขึ้นแค่ไหน ซึ่งการที่เราค่อย ๆ ทดสอบการเปลี่ยนแปลงทีละน้อย ๆ ทำให้ทั้งสองฝ่ายคือ Development ที่ผลิต Features และ ฝ่าย Operational ที่ทำหน้าที่การันตีรับรองการใช้งานให้มีความ High Available ไปตาม SLA ที่กำหนดไว้เข้าใจกันมากขึ้นอีกด้วย



กระบวนการแบบ DevOps (12)

เพราะปัญหาของการทำงานโดยแยกเป็นฝ่ายใครฝ่ายมันนั้น ทำให้เราไม่สามารถทราบถึงปัญหาเหล่านี้ได้เลยว่าซอฟต์แวร์ที่พัฒนาขึ้นมา นั้นมี Bugs ที่เกิดจากการพัฒนาซอฟต์แวร์ของ developer เองหรือเกิดจาก Infrastructure ที่ไม่รองรับการ Breaking Change ใหม่ ๆ ของซอฟต์แวร์ ที่ต้องการซอฟต์แวร์ หรือ OS ใหม่ ๆ มาทำงานร่วมกันกับซอฟต์แวร์ที่พัฒนาขึ้นมา กันแน่

ดังนั้นหลังจากที่เราเข้าใจถึง Context และข้อเสียจากการทำงานแบบ Silo Based ไปแล้วทีนี้เราจะมาดูกันว่าแล้วเรื่องราวกับแนวคิดใดบ้างที่การจะมุ่งไปสู่คำว่า DevOps มีอะไรบ้าง



กระบวนการแบบ DevOps (11)

วัฒนธรรมการคิดของ DevOps



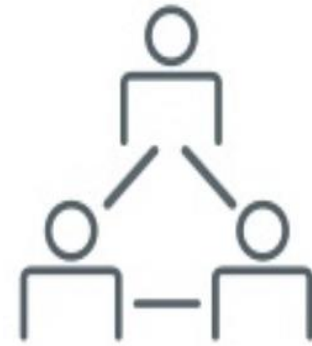
Culture



Automation



Measurement

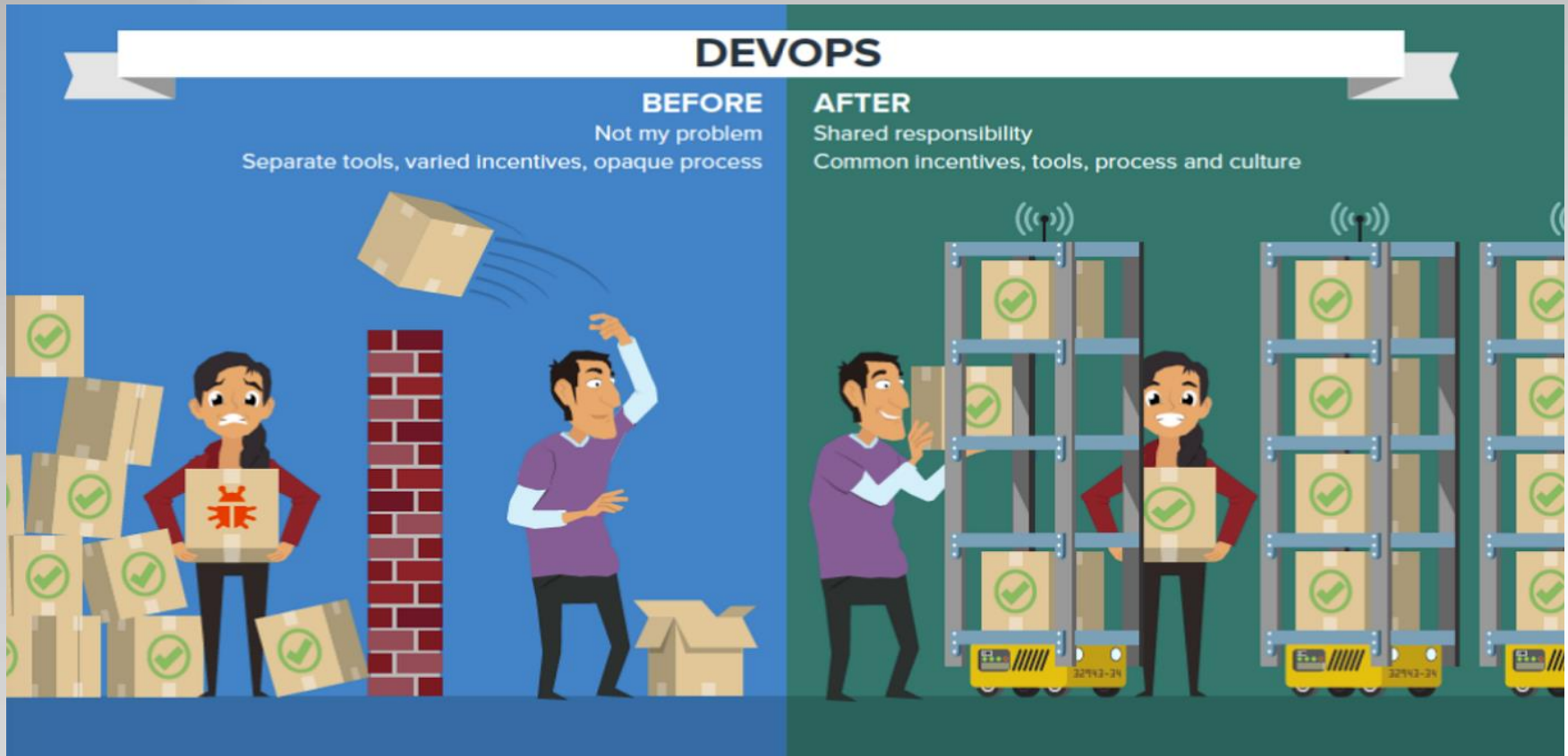


Sharing



กระบวนการแบบ DevOps (12)

1) Culture คือเรื่องของวัฒนธรรมในองค์กร





กระบวนการแบบ DevOps (13)

1) Culture คือเรื่องของวัฒนธรรมในองค์กร

ซึ่งเราต้องมีทีมที่เห็นคุณค่าของแนวคิดของ DevOps ก่อนว่ากระบวนการนำ DevOps มาใช้สามารถช่วยให้องค์กรสามารถ Delivery Product ได้เร็วขึ้น ซึ่งตรงนี้จะไม่ใช่เพียงแค่การที่เรานำคำศัพท์ใหม่ๆ ใหม่ๆ อย่าง Automation, Zero Trusted Architecture, GitOps และอื่นๆอีกมากมายที่เกิดขึ้นอยู่ทุกวันนี้มาพูดใช้กันเฉยๆ เพราะกระบวนการทำ DevOps คือการที่ทั้งทีม IT และทีม Business ตลอดจนไปถึงระดับโครงสร้างของผู้บริหารเห็นชอบกับแนวคิดของการประสานงานเข้าไปด้วยกันซึ่งจะทำให้ช่องโหว่ระหว่างแต่ละทีมค่อยๆหายไป



กระบวนการแบบ DevOps (14)

เพราะด้วยแนวคิดของการแบ่งแยกทีมแบบ Silo Based ที่เชื่อว่าแต่ละคนควรรับผิดชอบเฉพาะงานของตนเองทำให้ผลงานที่ออกมาขาดความเป็น Ownership ของ Product โดยองค์กรรวมแต่จะรับผิดชอบเฉพาะงานที่ตนทำเท่านั้น ซึ่งเมื่อเราย้อนไปดูเรื่องของ Product Digital Platform เราไม่สามารถนำแนวคิดที่ว่าแต่ละคนจะแก้ปัญหามเฉพาะจุดของตนเองมาใช้ได้ เพราะปัญหานั้นไม่ได้เกิดจาก Single Root Cause เท่านั้นแต่ทุกอย่างภายในระบบ IT ล้วนมีความเกี่ยวข้องกันตั้งแต่ Business Requirement, Program, Network, Database และ Infrastructure ดังนั้นถ้าหากเกิดปัญหาขึ้นมาแล้วแต่ละคนไม่เห็นภาพรวมของ Product ไม่เข้าใจ Flow การทำงานของระบบ การแก้ไขก็จะขึ้นอยู่กับคนๆเดียว หรือคนเฉพาะกลุ่มเท่านั้น



กระบวนการแบบ DevOps (15)

ซึ่งทำให้กลับมาเป็นปัญหาดังภาพคือ แต่ละคนสร้างสินค้าเพื่อทดลองและก็โยนไปให้อีกฝ่าย โดยไม่สนใจว่าปัญหาระหว่างการขนส่งนั้นเป็นอย่างไร ตรงจุดนี้คือจุดหลักเลยที่เป็นปัญหาในหลายๆองค์กร ก่อนที่จํานำ DevOps หรือ Cloud Native Application ไปใช้เพื่อ Transform Monolith แต่ยังมีปัญหากับวัฒนธรรมในองค์กร ที่ไม่เอื้ออำนวยต่อการเกื้อกูลกัน โดยเฉพาะ Technology ใหม่ๆที่เข้ามาอย่างต่อเนื่องถ้าหากเรายังไม่สามารถผสานทั้งสองทีมเข้าด้วยกันได้ การเรียนรู้เพื่อไป Cloud Native Technology จะไม่สามารถเกิดขึ้นได้เลย



กระบวนการแบบ DevOps (16)

2) Automation

หลังจากที่เราได้เห็นแนวคิดของทัศนคติและวัฒนธรรมองค์กรไปแล้วว่าทำไม DevOps ไม่ใช่เรื่องที่แค่การนำคำศัพท์ Technology ใหม่ๆ เข้ามาลง แต่ต้องเกิดจากทัศนคติที่ดีด้วย โดยในข้อต่อมาจะเป็นเรื่องของ Technical ซึ่งนำมาแก้ Pain point คือเรื่องของ Process การ Manual Configuration เพื่อจะเตรียม Infrastructure และการทดสอบ Code ว่ามีปัญหาหรือไม่ซึ่งสังเกตดูว่าขั้นตอนเหล่านี้ล้วนสามารถทำเป็น Automation ได้



กระบวนการแบบ DevOps (17)

โดยมีแนวคิดที่ว่าเมื่อใดก็ตามที่ Developer Push Code ไปยัง Version Control แล้วเราจะทำให้มี Trigger ผ่าน Webhook ไปเรียกยัง Continuous Integration Pipeline ของเรา ซึ่งตัว Continuous Integration (CI) Pipeline จะทำหน้าที่ทดสอบว่า Code ของเราผ่าน Test หรือเปล่าและมีการ Scan Quality ของ Code ต่างๆถ้าหากไม่ผ่านมาตรฐานก็จะแจ้งกลับไปยัง Developer ให้กลับไปแก้ Code อัตโนมัติซึ่งตรงจุดนี้ทำให้เกิดสิ่งที่เรียกว่า Fast Feedback Loop ทำให้สามารถเรียนรู้จากจุดบกพร่องได้เร็วขึ้นและเพิ่มประสิทธิภาพของ Code ที่จะออกไปอีกด้วยซึ่งตรงจุดนี้จึงเป็นการหักล้างความเชื่อว่า Automation แล้วคุณภาพของ Code จะลดลงหรือเปล่า ?

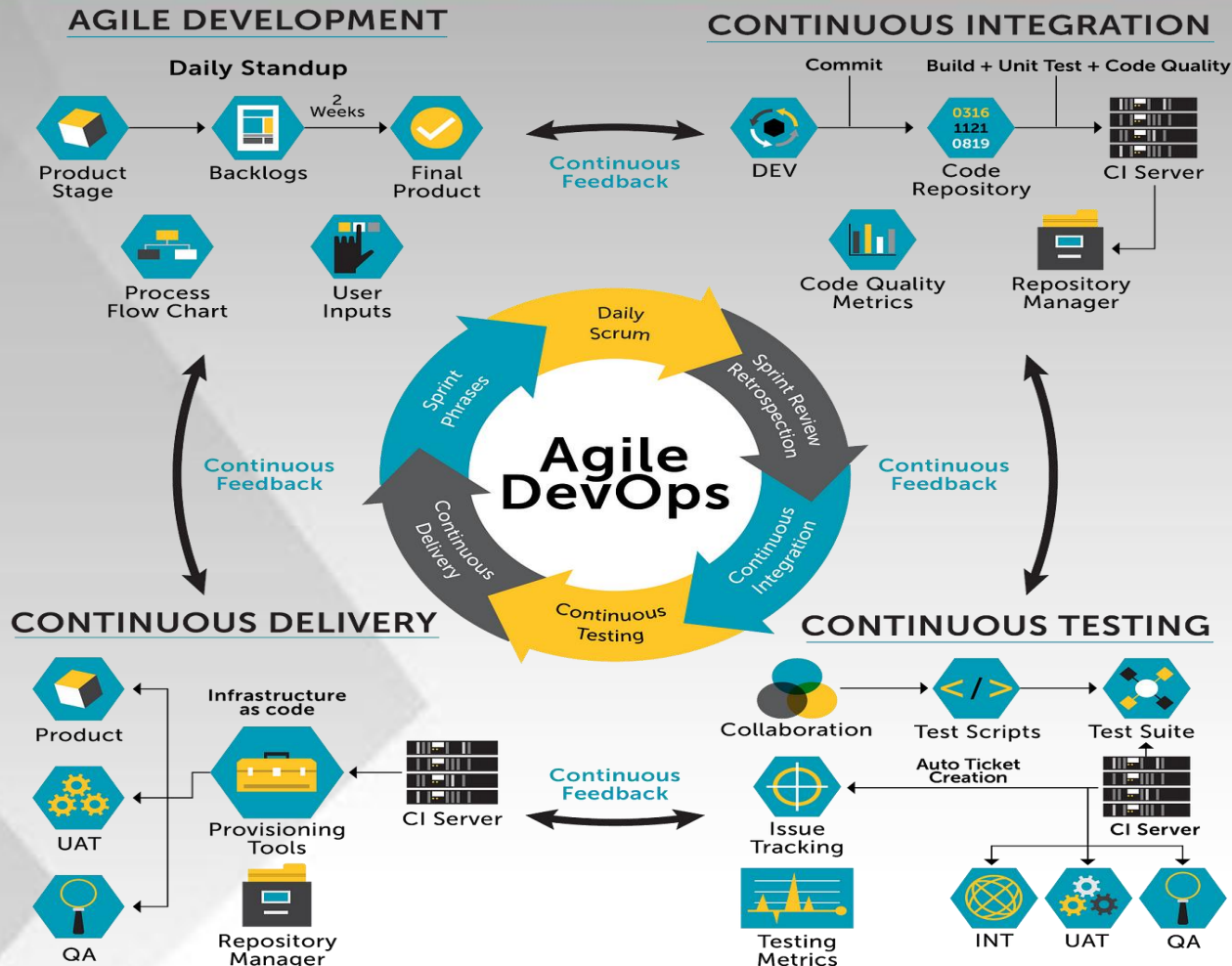


กระบวนการแบบ DevOps (18)

คำตอบก็คือการทำ Automation ไม่ได้ทำให้ Defect หลุดออกไปมากขึ้น เพราะถ้าเราผ่านเรื่องของ Culture ที่ดีในการเขียน โปรแกรมและความเป็นเจ้าของ Product มาแล้วเราจะทำการทดสอบ โปรแกรมของเราอยู่เสมอ ดังนั้นการมี Unit Test + Integration Test ผังใน CI จะทำให้เราได้ Feedback Loop ที่รวดเร็วและเสมือนมีการ Review Code ไปในตัวอีกด้วย



กระบวนการแบบ DevOps (19)





กระบวนการแบบ DevOps (20)

โดยจากภาพจะเห็นว่าในส่วนของ Cultures ก็จะเป็นส่วนของ Agile Development และ Automation จะเริ่มที่ Continuous Integration + Continuous Testing เพื่อทดสอบปัญหาต่างๆและมากไปกว่านั้นเรื่องของ Security พื้นฐานเองก็เป็นสิ่งที่ต้องคำนึงถึงดังนั้นปัจจุบันคำว่า DevOps จึงถือว่าตกยุคไปแล้วแต่กลายมาเป็น DevSecOps ซึ่งเพิ่มเรื่องของ Security เข้ามาด้วยเพราะด้วยยุคของ Cloud Native สิ่งที่มาคือ Attack Surface ที่เพิ่มมากขึ้นตามไปด้วยเพราะด้วยหลาย Endpoint ของ Tool หรือ Service ที่เกิดมากขึ้นทำให้การคำนึงถึง Security Endpoint ถือเป็นเรื่องที่สำคัญเช่นกันแต่สำหรับ tools นั้นก็มีให้เลือกมากมายในขั้นของ CI ไม่ว่าจะเป็น Jenkins, CircleCI, GitlabCI และอื่นๆอีกมากมาย



กระบวนการแบบ DevOps (21)

ซึ่งระหว่างการทำ Integration จึงควรเสริมเรื่องของการ Scan Security หรือ Vulnerable ที่มีโอกาสเกิดขึ้นกับ Source Code เราไปด้วย (ลองสังเกตดูว่า Github จะมีการเตือนเรื่อง Dependency มีช่องโหว่และเตือนให้เราทำการอัปเดต แต่ก็ต้องดูด้วยนะครับว่ากระทบต่อระบบทั้งหมดหรือเปล่าหากอัปเดตแล้ว ไม่อย่างนั้นอาจจะสร้างปัญหาเพิ่มขึ้นแทน)



กระบวนการแบบ DevOps (22)

ในส่วนต่อมาก็คือ Continuous Delivery จะเป็นเรื่องของการที่เราจะสามารถนำ Software, Dependency หรือ Product ใดๆต่างๆของเราไปให้ปลายทางแก่ User ได้อย่างไร ซึ่งที่เรามองคำว่า Delivery Software, Dependency และ Product ไปกลุ่มเดียวกันนั้นเพราะว่าการ Delivery ไม่ได้จำกัดแค่การนำ Application สำหรับ End User ไปขึ้น Server แล้วมี Features ใหม่มาเท่านั้นแต่ มันหมายถึงการส่งมอบใดๆก็ตามที่จำเป็นต้องใช้ใน System ทั้ง Platform ยกตัวอย่างเช่นก่อนที่เราจะมี Application ได้เราก็ต้องมี Infrastructure ก่อนซึ่ง ในยุคของ Cloud Native เราก็มีย Cloud Provider มากมายไม่ว่าจะเป็น AWS, Azure, Google Cloud หรือกระทั่งของประเทศไทยอย่าง Nipa.Cloud



กระบวนการแบบ DevOps (23)

แต่ในขั้นตอนการ Provisioning เครื่อง Virtual Machine เหล่านี้บน Cloud เอง ก็ยังต้องเสียเวลาในการ Configuration อยู่ดีไม่ว่าจะเป็น Network, Spec Hardware หรือ Access Control ของบัญชีองค์กรต่างๆ ซึ่งถ้าหากเราต้องการ Deploy Application จำนวนมากและ Scaling เพื่อรองรับผู้ใช้จำนวนมากที่เพิ่มขึ้นในช่วงเวลาสั้นๆ อย่างระบบ ไทยไม่ทิ้งกัน, ลงทะเบียน GAT/PAT ก็คงไม่สามารถไปเพิ่ม Provisioning เครื่อง VM บน Cloud ทีละเครื่องแล้วมา Config Network ทีละจุดได้เพราะคงเสียเวลามากมายและมีโอกาสผิดพลาดอีกด้วยเพราะต้องเข้าไปติดตั้ง Software เองทีละจุด สิ่งที่เกิดขึ้นคือเรื่องของ Security ที่มีโอกาสถูก Exposed มากขึ้นหากต้องใช้แรงงานคนไป Config เชื่อมต่อกับ Database ต่างๆ



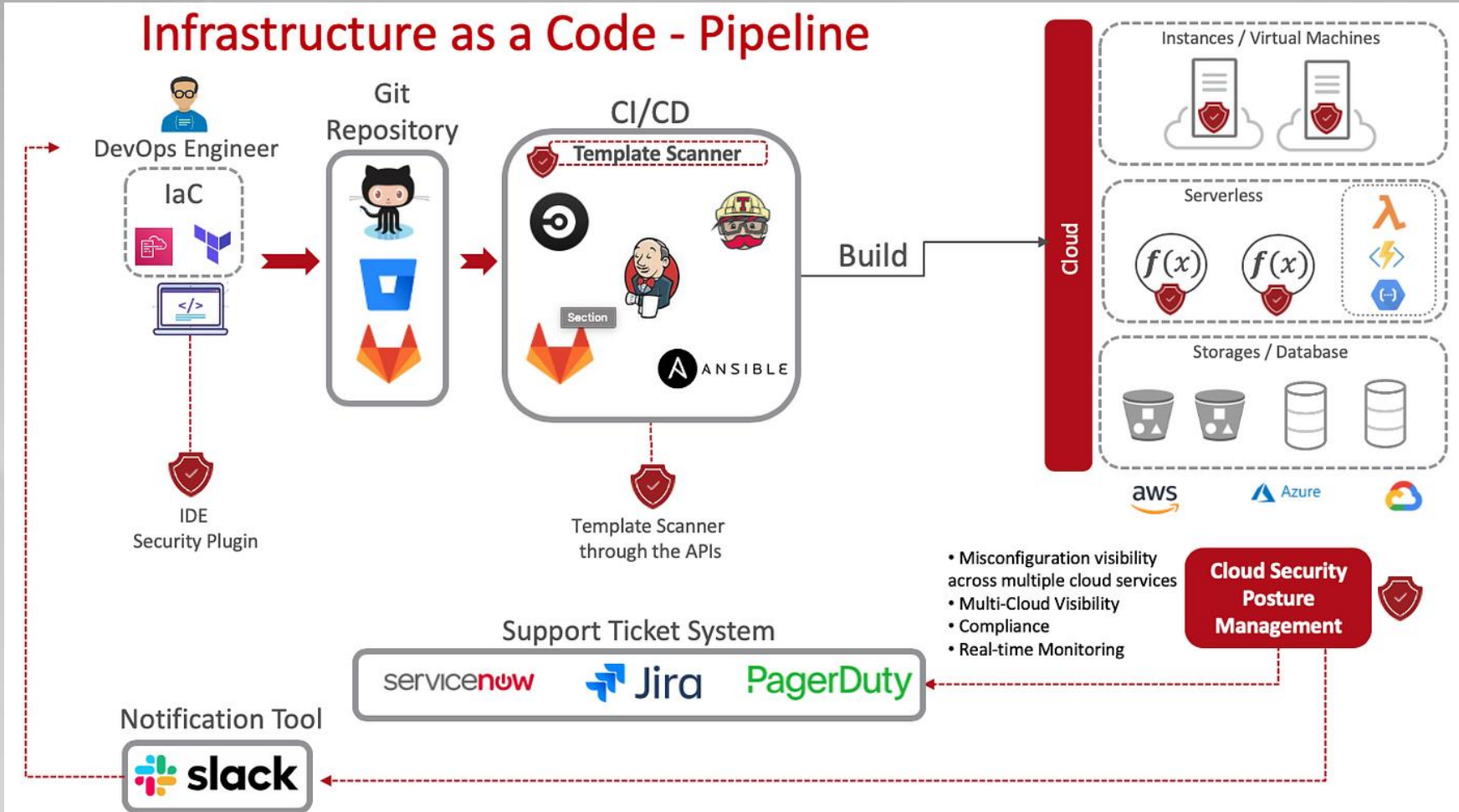
กระบวนการแบบ DevOps (24)

Password ก็มีโอกาสรั่วไหลไปได้ หรือการ Scale เครื่องให้เล็กลงหากผู้ใช้ลดลง ก็ต้องมาปิดทีละเครื่องก็เป็นการเสียเวลาเช่นเดียวกัน ดังนั้นถ้าเรามองลึกลงไประดับของ Continuous Delivery ของ System Platform นั่นก็คือการทำอย่างไรให้เราสามารถ Provisioning เครื่อง Virtual Machine จำนวนมากๆ พร้อมๆกันได้เป็นหลักหมื่นเครื่องในระยะเวลาไม่ถึง 10 นาที พร้อมกับติดตั้ง Application ของเราลงไปบนเครื่องเหล่านั้นด้วยเมื่อ Server ทำการ Start สำเร็จแล้ว



กระบวนการแบบ DevOps (25)

Infrastructure as a Code - Pipeline





กระบวนการแบบ DevOps (26)

โดยจากภาพจะเห็นว่าในส่วนของ Cultures ก็จะเป็นส่วนของ Agile Development และ Automation จะเริ่มที่ Continuous Integration + Continuous Testing เพื่อทดสอบปัญหาต่างๆและมากไปกว่านั้นเรื่องของ Security พื้นฐานเองก็เป็นสิ่งที่ต้องคำนึงถึงดังนั้นปัจจุบันคำว่า DevOps จึงถือว่าตกยุคไปแล้วแต่กลายมาเป็น DevSecOps ซึ่งเพิ่มเรื่องของ Security เข้ามาด้วยเพราะด้วยยุคของ Cloud Native สิ่งที่มาคือ Attack Surface ที่เพิ่มมากขึ้นตามไปด้วยเพราะด้วยหลาย Endpoint ของ Tool หรือ Service ที่เกิดมากขึ้นทำให้การคำนึงถึง Security Endpoint ถือเป็นเรื่องที่สำคัญเช่นกันแต่สำหรับ tools นั้นก็มีให้เลือกมากมายในขั้นของ



กระบวนการแบบ DevOps (27)

คำตอบนั้นก็คือการที่ให้ Continuous Delivery ของเราไม่ได้จำกัดแบบ Application แต่เจาะลึกไปถึง System Platform ที่เริ่มตั้งแต่ให้ Infrastructure ถูกเก็บอยู่ในรูปแบบของ Infrastructure as a Code (IaC) เช่น Terraform จะทำให้เราสามารถ Provisioning Resource ไปยัง Cloud Provider ได้อย่างง่ายดาย และลดปัญหาเรื่อง State ของ Server บน Cloud Server คลาดเคลื่อนกันเพราะการ Config ที่ผิดพลาด แต่เมื่อมี IaC มาใช้งานจะทำให้เราสามารถ Tracking ความถูกต้องล่าสุดของ Cloud Resource ที่เรามีทั้งหมดได้



กระบวนการแบบ DevOps (28)

ซึ่ง Tools เหล่านี้ล้วนเชื่อมเข้ากับ Version Control เหมือนทางฝั่ง Developer เช่นกัน ทำให้ลดปัญหาของ Operational Team ที่ทำหน้าที่ดูแล Infrastructure ไม่สามารถทดลอง Provisioning Server หรือการแก้ปัญหา Configuration Drifting บน Production Environment หลังจากที่เราสามารถจัดการเรื่องของการ Provisioning แบบ System to System ได้แล้วเราก็จะทำการติดตั้ง Software Infrastructure ที่จำเป็นไม่ว่าจะเป็น Docker, Jenkins ผ่าน Ansible ซึ่งก็เป็น IaC ตัวหนึ่งซึ่งเหมาะกับการติดตั้ง Software และเมื่อขึ้นมาที่ระดับ Layer ข้างบนระดับ Software Layer เราก็สามารถนำ Source Code ที่ Build เป็น Artifact พร้อม Deploy ส่งไปให้กับ Continuous Deployment Pipeline ของเราเพื่อทำการ Deploy Application ต่อไป



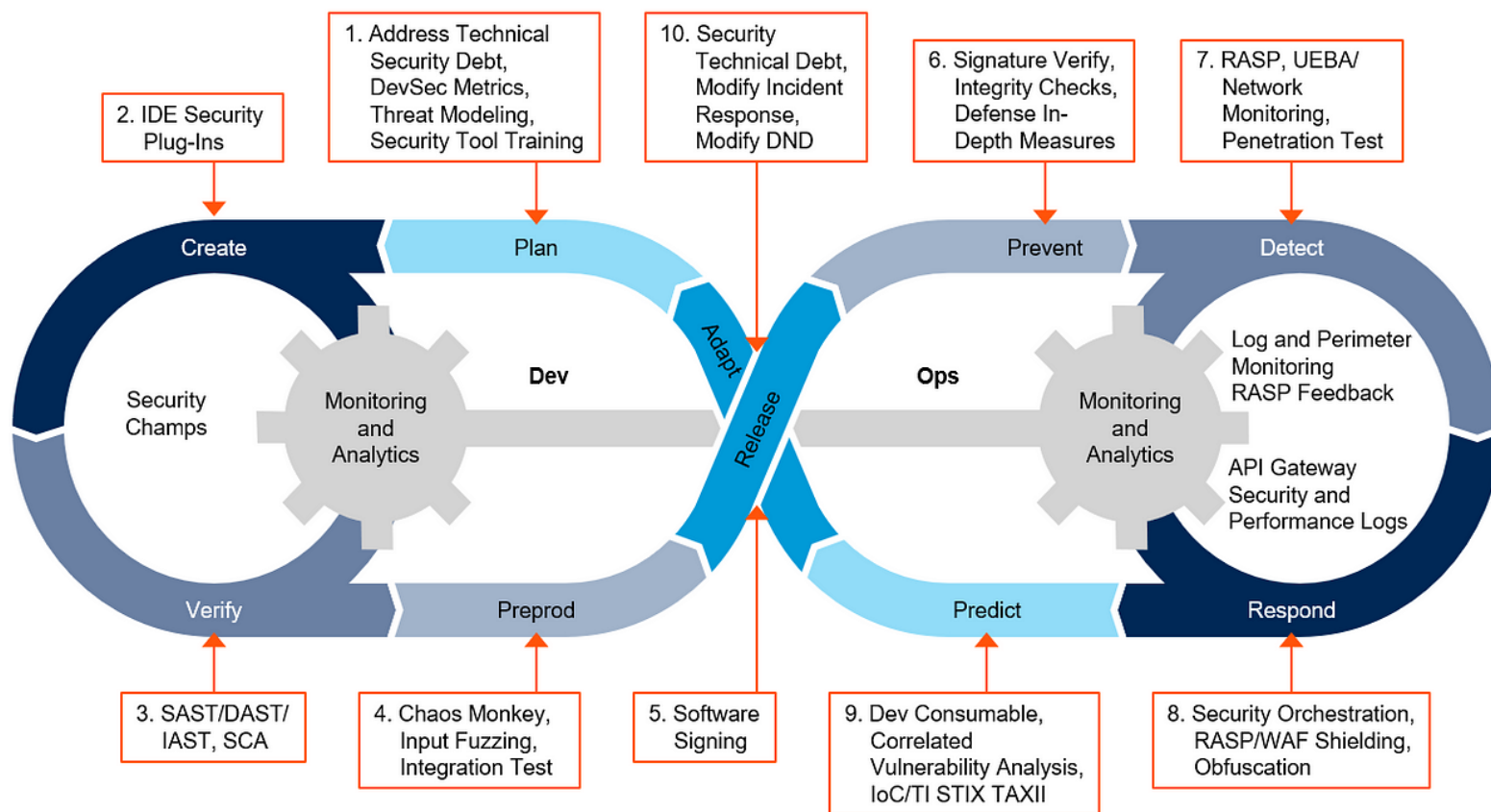
กระบวนการแบบ DevOps (29)

เมื่อต้องการใช้งานควบคู่กับกับ Scaling Infrastructure ผ่าน tools IaC อย่าง Terraform ของเรา และสามารถรับ Feedback ครบทั้ง Pipeline CI/CD ได้ผ่านระบบ Support Ticket System ที่ให้ User สามารถกลับมา feedback ถึงปัญหา หรือ Bugs ที่เราต้องแก้ไข ก็จะแสดงอัตโนมัติผ่านหน้า Dashboard เหล่านั้น ทำให้ Team ทราบได้ว่าระบบกำลังมีปัญหาใดที่ต้องทำการ Tracking กลับไปบ้าง



กระบวนการแบบ DevOps (30)

The DevSecOps Toolchain



Source: Gartner
ID: 377293



กระบวนการแบบ DevOps (31)

หลังจากที่เราเห็นในเรื่องของ DevOps Pipeline ไปแล้วเราก็จะมาเสริมในส่วนสุดท้ายคือ Security ใน Flow ของ DevSecOps ซึ่งอาจจะมองว่าเป็นกำแพงใหญ่ของการทำ DevOps เพราะในทุกๆ Enterprise ใหญ่ๆ ล้วนต้องมี Framework สำหรับการทำ Audits ด้วยกันทั้งนั้นเพื่อตรวจสอบความถูกต้องต่างๆ และอะไรจะทำให้ Enterprise สามารถนำ Cloud Native Technology ซึ่งเป็นยุคของ Open Source Project กันทั้งนั้น ในมุมมองของ Security Team เองก็จะมองว่าไม่ผ่านแน่ๆ เพราะว่าเต็มไปด้วยช่องโหว่จำนวนมากที่อาจจะถูกโจมตีได้ ซึ่งกำแพงเหล่านี้คงไม่มีทางหายไปหากเรายังมองว่า Security Audit ก็ยังเป็นอีกทีมหนึ่ง



กระบวนการแบบ DevOps (32)

ดังนั้นการรวมกันเพื่อให้ผ่าน Process นี้ไปก็คือนำทีม Security เข้ามาใน Process ระหว่างที่เรากำลัง Build Software ของเราด้วยเช่นกันทำให้เขาทราบหมดว่าระบบของเรานั้นมีช่องโหว่ใดบ้างหรือเปล่าดีกว่าให้ Team Security ไปนั่งหาเองตั้งแต่ต้น ซึ่งการนำ Security มาใช้กับ DevOps นั้นถือเป็นสิ่งที่ถูกต้องและจำเป็นด้วยซ้ำไปเพราะ DevOps คือการทำให้ทุกคนในทีมเห็นคุณค่าของ Product เพื่อส่งมอบ Software ที่มีคุณภาพออกไปจนจบ Flow การทำงาน



กระบวนการแบบ DevOps (33)

ดังนั้นหากเราสามารถทำทีม Team Security มาช่วยในเรื่องของการแบ่งปันความรู้พื้นฐานอย่าง OWASP Top Ten หรือการ Monitoring ต่างๆ ที่มี Tools ออกมามากมายอย่าง Elasticsearch Stack ที่ขยายไปจนถึงระดับ Security Incident Event Management (SIEM) และล่าสุดก็รองรับเรื่องของ Security End Point Management พร้อมกับการทำ Centralize Logs เพื่อทำตาม Compliance ต่างๆ ไม่ว่าจะเป็น GDPR, HIPAA ก็มี Tools ให้พร้อมแล้วเช่นกัน พร้อมกับการนำ Security Scanner เข้ามาทดสอบโปรแกรมตั้งแต่ในช่วงของ CI Pipeline อย่าง ZAP OWASP เองก็ทำได้เช่นกัน

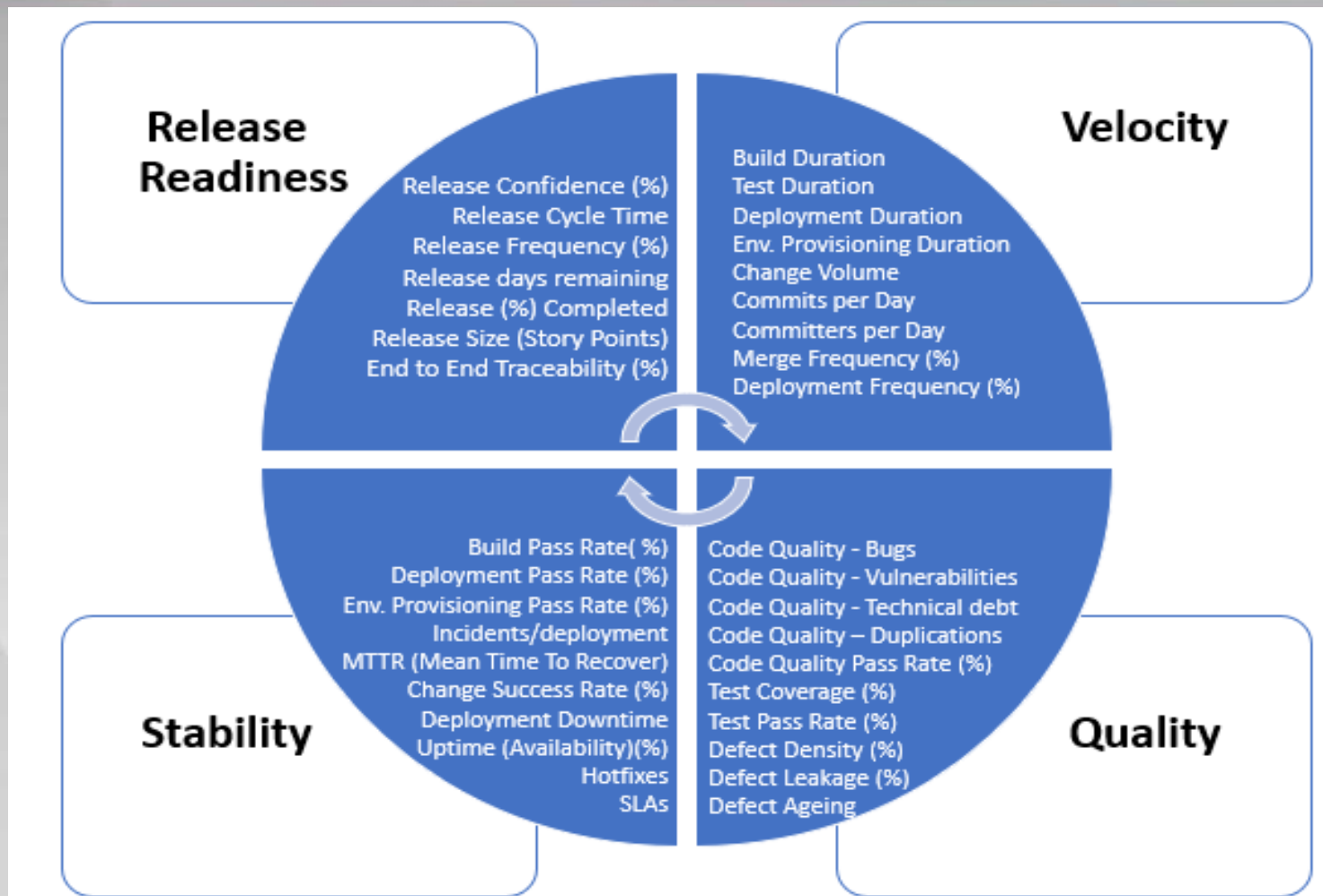


กระบวนการแบบ DevOps (34)

3) Measurement คือขั้นตอนของการวัดผลว่า Feedback Loop ที่เราได้รับมานั้นช่วยให้ Process การทำงานของทีมดีขึ้นไหมซึ่งหลังจากที่เรามีเรื่องของ Technical ครบแล้วในข้อที่สองสิ่งที่จะทำให้ Process ของเราดีขึ้นนั้นก็คือการประเมินผล แต่สิ่งหนึ่งที่เราต้องเข้าใจคือการประเมินของเรานั้นจะไม่ใช่การจับผิดใครหรือการ โทษไปที่ว่าใครผิดเพราะแนวคิดของ DevOps คือการ Blameless ที่เราจะไม่โทษที่บุคคลคนใดคนหนึ่งเพราะทั้งหมดคือ Process ที่เราช่วยกันทำมาด้วยกัน แต่เราจะมาดูที่เราสามารถมีวิธีใดเพื่อใช้ในการแก้ไขปัญหาได้บ้าง โดยปัจจัยที่เราสนใจหลัก ๆ ก็อาจจะเป็น Mean Time to Recover (MTTR) หรือช่วงเวลาที่ใช้ในการกู้คืนระบบเมื่อเกิด Failure เกิดขึ้น รวมไปถึง Code Quality ที่ส่งมาจาก Pipeline CI/CD ก่อนหน้า



กระบวนการแบบ DevOps (35)





กระบวนการแบบ DevOps (36)

ซึ่งถ้าเราดูจากภาพเราจะเห็นทั้งสี่มุมมองคือ Release Readiness ที่หมายถึงความพร้อมในการปล่อย Product เราออกไปตั้งแต่ระดับเล็กๆจนไปถึงส่วนของ End to End และก็มีส่วนของ Velocity ที่วัดความรวดเร็วในขั้นตอนระหว่างการ Coding วันหนึ่งมี Commit อัปเดตเข้าไปยัง Version Control เท่าใด มีการ Build Artifact เท่าไหร่ และนำไป Deploy ที่ Staging Server เพื่อทดลองก็ครั้งเป็นต้น ต่อมาก็จะเป็นเรื่องของ Stability ว่า Software ของเราเสถียรมากแค่ไหน สามารถรับ Fault Tolerance ได้มากเพียงใดถ้าหากเกิด Failure จะมีการกู้คืนด้วยวิธีใดบ้างใช้เวลาเป็นเท่าไรซึ่งทาง Tech Company ใหญ่ๆอย่าง Netflix เองก็จะมีการทดสอบเรื่องของ Fault Tolerance เหล่านี้บน Production Stream



กระบวนการแบบ DevOps (37)

หน้าที่เราดูกันทุกวันอยู่ด้วยซึ่งมีแนวคิดของเรื่องการทำให้ Production Server เกิด Fault ขึ้นด้วยการปิด Server หรือ Shutdown Service บางตัวลงเพื่อทดสอบว่า ระบบของต้นนั้นสามารถการันตี Uptime ได้เท่าไรกันจริงๆกัน แน่ในขณะที่มี Users เข้ามาใช้งานจริงๆอยู่ด้วย และในส่วนสุดท้ายก็จะเป็นเรื่องของ Quality Control เพื่อให้มั่นใจว่า CI/CD Pipeline ของเรานั้นไม่ได้ปล่อยสิ่งที่ Defect ออกไปแก่ผู้ใช้ปลายทางการทดสอบจึงเป็นหัวใจสำคัญที่ไม่ได้ตกไปอยู่ที่ Tester ที่มออย่างเดียวแต่ต้องเริ่มตั้งแต่ระดับจุดเล็กสุดที่คนเข้าใจก็คือ Programmer ที่เป็นคนเขียน Unit Test นั้นเอง รวมไปถึงการคำนึงถึง Security ต่างๆก็ควรจะมีด้วยเช่นกัน



กระบวนการแบบ DevOps (38)

4) Sharing

จากทั้งสามข้อของวัฒนธรรมการคิดแบบ DevOps และเรื่องของ Trending Technology ที่เกิดขึ้นอย่างรวดเร็วเราคงทราบแล้วว่าถ้าหากภายในทีมเราไม่มีการแบ่งปันความรู้กันคงเป็นไปได้เลยที่คนๆเดียวจะสามารถเข้าใจทำทุกอย่างได้ด้วยตัวคนเดียว โดยไม่สอบถามไม่เรียนรู้ Best Practice จากทีมอื่น หรือถึงทำได้ตัวคนเดียวก็คงจะเสียเวลาและช้ามากแถมยังเป็นการใช้เวลาไปกับสิ่งที่ไม่จำเป็นอีกด้วยเพราะเรามีคนที่เคยผ่านขั้นตอนหรือปัญหาเดิมๆ เหล่านั้นอยู่แล้ว เราก็ควรนำวิธีการแก้ปัญหาเหล่านั้นมาแชร์กันเลยเพื่อให้เรียนรู้ได้เร็วขึ้นโดยไม่เสียเวลาไปกับปัญหาเดิมๆอีก



กระบวนการแบบ DevOps (37)

Traits of the Cloud Natives



Customer-centric



Learning culture



Agile development



Focus on products,
not projects



Automation of entire
delivery process



Heavy use of open-
source software





กระบวนการแบบ DevOps (38)

Silo Team-Based ในตอนต้น แต่เมื่อใดก็ตามที่คนอื่นๆ มีความรู้กันแบบ Cross-Functional Team อันหมายถึงทุกคนในทีมเข้าใจ Business, Code, Infra, Process, Security แต่ก็มีหน้าที่หลักก็คืองานของตนที่รับผิดชอบและ ถนัดจะทำให้เวลาแก้ปัญหาทุกคนเห็นภาพรวมเดียวกัน และนำความสามารถ ที่ตนถนัดเข้าไปวิเคราะห์แก้ปัญหาในส่วนอื่นที่เกิดในระบบได้ ไม่จำกัดแค่ ส่วนที่ตนเองรับผิดชอบเท่านั้นซึ่งการ Sharing เหล่านี้ก็จะไปผนวกกับการ ทำงานแบบ Agile ที่มีความคล่องตัวอีกด้วยพร้อมกับการทำให้ทุกคนรู้สึกเป็น ส่วนหนึ่งของทีมจริงๆ ไม่ใช่เข้ามาทำงานไปวันๆ เพื่อให้จบโปรเจกต์ตามที่ มอบหมาย แต่มันคือเราเป็นส่วนหนึ่งของเจ้าของ Products เรารู้สึกว่า Software ที่เราสร้างนั้นมีคุณค่า



กระบวนการแบบ DevOps (39)

ดังนั้นเราจะเห็นแล้วว่าการมุ่งสู่ DevOps ขององค์กรนั้นไม่ใช่แค่การนำ Technology ใหม่ ๆ เข้ามาเพราะถ้านำเข้ามาแต่ Technology แต่คนในองค์กรยังไม่ทราบถึงเหตุผลและแรงจูงใจสุดท้ายก็ไม่มีใครใช้ แถมยังทำให้มีโอกาสผิดพลาดสูงมากขึ้นเพราะการใช้ Technology ใหม่ ๆ และไม่เข้าใจก็ทำให้มีโอกาสผิดพลาดจาก Bugs หรือเกิด Vulnerable แต่ในทางกลับกันหากเข้าใจถึงแก่นแท้ของ DevOps องค์กรจะสามารถลดต้นทุนไปได้มากเพราะมีทั้งการ Optimize Software ที่ดีขึ้น การเลือกใช้ Open Source Tools ต่าง ๆ หลากหลายตัวที่มาช่วยให้งานง่ายขึ้นด้วย



แหล่งข้อมูล

- Gitlab CI/CD - Gitlab
- Get started with GitLab CI/CD - Gitlab
- GitLab Runner – Gitlab
- <https://www.codium.co/blogs/>
- - cmswire. (2018). 5 Signs Your Organization Is Too Siloed. <https://www.cmswire.com/.../5-signs-your-organization-is.../>
- - oknation.nationtv.tv. (2561). ทลายการทำงานแบบไซโล!!!
<http://oknation.nationtv.tv/.../pigkys.../2018/04/04/entry-1>
- - ประชาชาติออนไลน์. (2561). องค์กรที่มีความยืดหยุ่น. <https://www.prachachat.net/facebook-instant.../news-182372>
- <https://wdrdres3qew5ts21.medium.com/>



เปิดโอกาสให้ซักถามครับ

Q & A